



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INFORMATION SYSTEMS

ÚSTAV INFORMAČNÍCH SYSTÉMŮ

CONFIGURABLE SYSTEM FOR USER INPUT TRANSFORMATION AND ROUTING

NASTAVITELNÝ SYSTÉM PRO ZPRACOVÁNÍ A SMĚROVÁNÍ UŽIVATELSKÝCH VSTUPŮ

MASTER'S THESIS

DIPLOMOVÁ PRÁCE

AUTHOR

AUTOR PRÁCE

Bc. JAN MALIŇÁK

SUPERVISOR

VEDOUCÍ PRÁCE

Ing. JAN PLUSKAL, Ph.D.

BRNO 2026

Master's Thesis Assignment



170821

Institut: Department of Information Systems (DIFS)
Student: **Maliňák Jan, Bc.**
Programme: Information Technology and Artificial Intelligence
Specialization: Bioinformatics and Biocomputing
Title: **Configurable System for User Input Transformation and Routing**
Category: Operating Systems
Academic year: 2025/26

Assignment:

1. Investigate existing methods and tools for capturing, transformation and routing of user input like keyboard and mouse events between devices.
2. Design a configurable modular system for capturing, transforming and routing user input events with a focus on low latency, extensibility, portability and security.
3. Implement the system as a cross-platform .NET application.
4. Design experiments to evaluate the functionality, performance and usability of the system.
5. Discuss the results of the experiments, compare the system with existing solutions and analyze possible areas for further development.

Literature:

- Jacko, Julie & Sears, Andrew. (2002). The Human-Computer Interaction Handbook: Fundamentals, Evolving Technologies, and Emerging Applications. ISBN 0805838384
- Dix, Alan & Finlay, Janet & Abowd, Gregory & Beale, Russell. (2004). Human-Computer Interaction. ISBN 9780130461094
- Corbet, Jonathan & Rubini, Alessandro & Kroah-Hartman, Greg. (2005). Linux Device Drivers. ISBN 9780596005900

Requirements for the semestral defence:

- points 1, 2 and partially point 3.

Detailed formal requirements can be found at <https://www.fit.vut.cz/study/theses/>

Supervisor: **Pluskal Jan, Ing., Ph.D.**
Head of Department: Kolář Dušan, doc. Dr. Ing.
Beginning of work: 1.11.2025
Submission deadline: 20.5.2026
Approval date: 30.10.2025

Abstract

The goal of this thesis is to design a system for the transformation of a user input based on context, combined with selective routing. This offers better utilization and unification of input devices over multiple machines. The desired outcome of this system is to increase the efficiency of multitasking, reduce the overhead of switching between mental contexts or between hardware devices, and increase productivity. Such a system should be highly customizable and easy to extend to allow a wide range of use cases. It should also be as independent of hardware and operating system as possible to allow interconnected control of multiple systems and simple migration to new setups. The thesis proposes how such a system can be designed and offers its own implementation. Finally, the implemented system is tested to evaluate its performance and usability.

Abstrakt

Cílem této práce je návrh systému pro transformaci uživatelského vstupu založené na kontextu, společně se selektivním směrováním. To umožňuje lepší využití a sjednocení vstupních zařízení pro vícero počítačů. Očekávaným výsledkem tohoto systému je zvýšení efektivity při práci na vícero úlohách, snížení nároků na přepínání mentálního kontextu i mezi fyzickými zařízeními, a zvýšení uživatelské produktivity. Takový systém by měl být vysoce přizpůsobitelný a snadno rozšiřitelný, aby mohl podporovat velmi široké možnosti využití. Měl by být také co nejvíce nezávislý od fyzického vybavení i od operačního systému, aby bylo možné jejich vzájemné ovládání a snadná přenositelnost na nové sestavy. Tato práce představuje, jak by takový systém měl být navržen a nabízí vlastní implementaci. Na závěr je implementovaný systém otestován pro ohodnocení jeho výkonnosti a použitelnosti.

Keywords

User input, control transformation, multitasking, productivity, context switching, modular system, scripting, performance, portability, .NET, Low Level Hooks

Klíčová slova

Uživatelský vstup, transformace ovládání, multitasking, produktivita, přepínání kontextu, modulární systém, skriptování, výkonnost, přenositelnost, .NET, Low Level Hooks

Reference

MALIŇÁK, Jan. *Configurable System for User Input Transformation and Routing*. Brno, 2026. Master's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Jan Pluskal, Ph.D.

Rozšířený abstrakt

Motivací k této práci byla snaha o dynamické ovládání vícero počítačových systémů pomocí jediné sady vstupních zařízení. Jako vhodný příklad zde může posloužit osoba s ADHD, která často mentálně přepíná mezi různými činnostmi, například mezi hraním videoher a programováním. V takovém případě by byl velmi užitečný nějaký způsob, jak toto přepnutí provést fyzicky stejně rychle, jak je toho schopna mentálně. To ovšem prakticky bývá možné jen zřídka, neboť je buďto potřeba přesunout ruce na druhou sadu vstupních zařízení, nebo je potřeba se přepnout na jiné okno. V obou případech může osoba ovládat jen jeden systém a nemůže okamžitě reagovat na události v druhém systému.

V této práci je proto nejprve proveden průzkum stávajících řešení tohoto problému. Důkladnější pohled je věnován programu AutoHotKey, systémům pro vzdálenou obrazovku, a různým alternativním metodám uživatelského vstupu. Protože tyto systémy neuspokojují plně požadavky pokrývající uvedený motivační příklad, je také zkoumáno, jak fungují zařízení pro uživatelský vstup, jaké jsou možnosti jejich přizpůsobení, a jak dochází k jejich interpretaci a následné reprezentaci v operačním systému.

Po důkladném zvážení potřeb a možností, byly určeny tyto požadavky na navrhovaný systém: bezpečnost, přenositelnost, rozšiřitelnost, selektivita, výkonnost. Je usuzováno, že těchto požadavků může nejlépe dosáhnout modulární systém, u něž je kladen velký důraz na uživatelovu možnost konfigurace tohoto systému v reálném čase. Následně jsou navrženy některé významné funkcionality, které by systém měl poskytovat. Aby se nebylo nutné omezovat na jediný operační systém, uživatelům vstup je co nejdříve po odchycení převeden na vlastní obecnou reprezentaci. Dále jsou prezentovány některé možnosti, kam lze transformovaný vstup směřovat, jak jej interpretovat a předat operačnímu systému.

Následně jsou navrženy způsoby, jak umožnit uživateli systém nastavit a kontrolovat. Jako základ je zvoleno ovládání přes příkazovou řádku, čehož lze využít i jako základ pro pokročilejší metody ovládání. Je popsán způsob rozšíření systému pomocí externích knihoven, metoda dynamického zřetězení komponent systému, a také je navržen skriptovací jazyk pro možnost bezpečného dodání sofistikované logiky transformace nebo chování systému. Dále je navrženo, aby systém řešil zabezpečení přenesením zodpovědnosti na uživatele, kdy mu budou poskytnuty relevantní data pro informované rozhodnutí. Pro detekci možných chyb a útoků jsou zdroje externích dat porovnávány s poskytnutými numerickými otisky, jenž jsou získány moderními hashovacími funkcemi. V případech, kdy zdroji otisku nelze plně důvěřovat, jsou data zašifrována. Zatímco tento přístup neznemožní čtení uložených dat třetí stranou, uživatel má vždy jistotu, že data, se kterými pracuje, jsou pravá a důvěryhodná. Pro zakrytí komunikace jsou data posílána přes síť také šifrována.

Pro vytvoření použitelného systému byl zvolen jako reprezentativní operační systém Windows, pro jehož vestavěnou podporu se používá globální nízkoúrovňové zachytávání a simulace uživatelského vstupu. Ostatní operační systémy sice nemají vestavěnou podporu, ale lze vytvořit alternativní komponentu v rámci externí knihovny a snadno tak rozšířit podporu i na další konfigurace. Aby systém mohl plnit základní požadavky bez dodání dodatečných funkcionalit uživatelem, je mimo samotné architektury programu dále také implementován překladač, který kód převede do snadno interpretovatelných struktur a také samotný interpreter. Pro snazší používání je také navržena podpora pro vytvoření uživatelského rozhraní, jehož popis je nezávislý zvolené platformě.

Že implementovaný systém umí splnit představené požadavky, byl otestován na několika exemplárních scénářích, kde dosáhl velmi dobrých subjektivních výsledků. Rovněž byly navrženy experimenty pro důkladné objektivní zhodnocení efektivity systému pro možnosti provádění vícero aktivit současně. Protože pro naučení se prakticky používat tento systém

je z principu potřeba dlouhá doba, jsou uvedeny jen výsledky subjektivního hodnocení. Výkonnost je zkoumána schopností systému zpracovávat velké množství vstupních událostí, kde dosahuje výborných výsledků, s odhadem průměrné doby zpracování pod 1 ms. Snadnost rozšíření systému byla vyhodnocena předáním komplexního problému velkojazykovému modelu skrze jediný dotaz, kde vygenerovaný kód vyžadoval jen velmi drobné úpravy aby plně podporoval požadovanou funkcionalitu.

Výsledkem je tedy systém, díky kterému lze ovládat více počítačů současně pomocí jediné sady vstupních zařízení. Systém umožňuje prakticky neomezené možnosti pro transformaci uživatelského vstupu, s několika užitečnými transformacemi již předpřipravenými, kdy tyto transformace lze aplikovat jen na konkrétní vstupní události nebo jen za konkrétních podmínek. Systém je dostatečně rychlý pro použití v časově kritických úlohách bez znatelného zpoždění. Systém také nabízí dostatečnou míru zabezpečení, aby bylo možné detekovat chyby v systému a aby klesly možnosti hackerského útoku. Díky své modulární architektuře a možnosti rozšíření pomocí externích knihoven a skriptů, je systém snadno rozšiřitelný a přizpůsobitelný pro širokou škálu možných použití, navíc je přenositelný na většinu běžných operačních systémů, protože většina funkcionality je postavena nad .NET technologií. Výsledný systém je dostupný na platformě GitHub¹ jako open source projekt.

¹<https://github.com/xBecanto/InputResender>

Configurable System for User Input Transformation and Routing

Declaration

I hereby declare that this Master's thesis was prepared as an original work by the author under the supervision of Dr. Jan Pluskal. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used ChatGPT and Le Chat to correct spelling and other language mistakes. I used GitHub Copilot when working on the software.

.....
Jan Maliňák
May 17, 2026

Acknowledgements

I would like to express my gratitude to my supervisor for the great opportunity of converting personal project into my Master's thesis, as well as for his suggestions that helped to improve it. Next, I would like to thank my family for all their support that allowed me to pursue my studies. I'm sending a big smiley face to my friend Kate for such needed emotional and mental help. A token of appreciation to Divi for all the calming and motivational words. Finally, to Silver, Lidar, and just all my friends, I owe you dessert, tea or a hug, your choice.

Contents

1	Introduction	2
2	User Input in Computer Systems	3
2.1	Input Devices	4
2.2	Low-Level Input Processing	7
2.3	High-Level Input Processing and Interpretation	10
2.4	Ergonomics and Human Factors	12
2.5	Existing Systems	13
3	Motivation and Design	17
3.1	Motivation	17
3.2	Input Transformation	21
3.3	Input Routing	22
3.4	Configuration and Control	23
3.5	Security	25
4	Implementation	29
4.1	Windows Input API	29
4.2	Scripting	31
4.3	Console Commands	34
4.4	Provided Input Transformations	35
4.5	User Interface	37
5	Evaluation	39
5.1	Performance Testing	40
5.2	Usability Testing	47
6	Conclusion	50
	Bibliography	52
A	Contents of the included storage media	55

Chapter 1

Introduction

Average computer workstation setup has evolved significantly over the past few decades. From a machine with only one keyboard and mouse with single display, we have now moved to multi-monitor setups with potentially multiple input devices or even multiple computers. Having a personal PC and a work laptop on the same desk is not uncommon. Additional devices such as tablets or smartphones are often present as well. This evolution brought big increase in computing capabilities and potential productivity, but also new challenges.

Individual devices are often designed to work independently, with little options for shared control. Switching between different systems will often require additional input hardware and physical movement. Even when using just a single computer, the delay caused by the need to switch between different applications can lead to a loss of focus and productivity. For some applications, such as gaming, the switching cost is often too high to consider any multitasking at all. Even such simple task as answering a quick message or writing down a short note can consist of multiple steps over multiple applications, potentially leading to loss of focus on the main task.

The goal of this thesis is to offer a system that could help to reduce these issues. Proposed system should simplify this input fragmentation by allowing flexible routing of user input from single set of input devices to multiple targets. To allow performing various tasks for targets, the system should also provide options for input transformations. Both routing and transformation options should be highly configurable to allow for wide range of potential use cases. Finally, the system should be as independent of hardware and operating system as possible to allow interconnected control of multiple systems.

Hardware and methods for user inputs are presented in Chapter 2. This chapter also explores how operating systems and applications can handle user input, and offers examples of existing solutions for input routing and transformation. Chapter 3 discusses the overall design of the proposed system and its individual components. Implementation details and specific solutions are presented in Chapter 4. Finally, Chapter 5 proposes methods for future evaluation of the system and discusses potential use cases.

Chapter 2

User Input in Computer Systems

In the early days of computing, a single keyboard was enough. Input was simple, direct, and bound to one machine. The arrival of the mouse brought precision, but also fragmentation, the need to alternate between typing and pointing. Today, as users navigate between multiple devices, operating systems, and input methods, this fragmentation extends even further. Maintaining seamless control now is less about hardware and more about orchestration.

To demonstrate this fragmentation, let us consider the following example. A person with a mild case of ADHD is used to a wide range of mental stimulations and switches between different activities very often. They are used to playing a video game for a few minutes, maybe a single online match, then switching to programming for about the same amount of time and back again after that. They feel that it helps not to get tired too fast due to the required long period of high mental concentration. But they wonder: Would it be possible to do these actions simultaneously? After some measurement, they notice that during the video game session, about 20 - 60% of the time no control of the game is needed. But the time segments are very small, only a few seconds. Normally, it would not be worth moving hands to another computer or switching to a different program window. If only this time could be reduced, for example by controlling the gaming and programming PCs from a single keyboard and mouse combination, it might be possible to use the gaming time much more effectively.

Expanding on this example, in video games there are many situations where it is not possible to control anything else, but the time is still not used effectively. This can be walking a long distance in the game or waiting for the next fight in a single match. In these situations, the player needs to be prepared for immediate action, but the waiting time could again be used better. The simple option to do this would be to write something with a mouse either using gestures or with something like a Morse code with left and right mouse buttons. The left hand remains on keyboard to allow control of the game while pressing a single key like Caps-Lock can switch the mouse back to control the game. A more advanced method could be to use a specific hardware such as a mouse with many keys, allowing one to control a game with a single hand, leaving the other hand for the programming.

This example can be seen as an extreme situation, but similar situations do arise from time to time. Staying in the realm of video games, at night it might be desirable to have the possibility of writing text messages to other teammates rather than speaking. A developer might often want to switch between a code editor, a window with documentation, and a virtual machine or console. These activities, along with many others, could be made more efficient by using a single keyboard and mouse combination to control multiple computers, simultaneously if possible.

2.1 Input Devices

Before taking a look at what was already created to deal with the scenarios above, either to eliminate the issue of input fragmentation or to boost productivity by helping to use existing input more effectively, it would be wise to define some criteria that an ideal system for user input should have. They do not have to be exhaustive, and it is expected that their importance can vary between different users and use cases, but it serves as a sufficient basis for the comparison of different solutions and to identify potential spaces for improvement.¹ These criteria are as follows:

- **Latency** — If there is a noticeable delay between user input and desired action, the system would not feel good for most tasks. For browsing the Internet, this delay probably has the greatest tolerance because the action itself, such as fetching and displaying a website, has a noticeable delay. For this, a delay in the order of hundreds of milliseconds is the upper limit for a usable system, whereas tens of milliseconds would be preferable. For text typing tasks such as programming, chat, or writing a research paper, a delay longer than a few tens of milliseconds interrupts the flow. As the most extreme requirement, any extra millisecond of delay is an issue for gaming.
- **Cost** — The higher purchasing price and potential maintenance costs, of course, contribute greatly to satisfaction of any given user.
- **System independence** — While systems designed specifically for one operating system or a hardware are possible and in use, for example, gaming consoles such as PlayStation Controller, Nintendo Switch, or HTC Vive. Often it is either due to integration of the said input device into the entire hardware system, or very limited compatibility between different hardware systems. In some scenarios, this can also be a deliberate limitation to help monopolizing owns industry. But normally this is an undesired property. Input systems should be plug-and-play wherever possible. The same goes for input processing software. The need to install additional software on the target device can vastly limit its usage.
- **Customization** — This is a typical problem when choosing input hardware. Someone might prefer a heavier mouse, while the other might prefer a lighter one. Keyboard layout can be a good example for this metric, with not only different general layouts but also with different options to input language specific characters.
- **Reliability** — Although this metric is more relevant for hardware design, software layer can still introduce potential points of failure like loosing packets over network, incorrect prediction of intended user action, and so on.
- **Precision** — An ideal device would produce exactly the input event that the user intended. With keyboards, some inaccuracy could be introduced by missing the desired key. Virtual keyboards often have some form of auto-correction that should fix these mistakes but can introduce new ones when writing some unknown word, for example. A mouse is typically precise in movement detection, but since the intention of a user was to click a button or draw a line, the nature of indirect control can complicate such tasks and therefore should also be treated as due to lack of precision.

¹These criteria are primarily based on personal experiences, discussions with other users, and general knowledge of the field of human-computer interaction [5, 26, 2].

- **Compactness** — A large device leads to less empty space that can be used for other useful purposes. It would also require either a lot of motion or force to control it. A typical keyboard with 4 rows has a limited compactness as a user cannot reach all keys without moving hands. The combination of keyboard and mouse would be considered even less compact as the user has to switch between holding the mouse and typing on the keyboard.
- **Multitasking** — The nice idea would be to have one single input device to control not only all personal devices (i.e. computers, smartphones, tablets, IOT connected devices, and more) but also all running processes inside them at will.
- **Intuitiveness** — Any input device becomes useless if it is way too difficult to learn how to use it. This metric can be visualized as a Learning curve, that is, a function showing usage efficiency, such as text written per minute or the time to move cursor onto desired UI element, based on the time spent learning.

It should be noted that the term *multitasking* does not represent here the idea of engaging multiple tasks of high mental demand and importance simultaneously. Such behavior can be very demanding and often leads to a significant decrease in the quality of work, among other issues. Two tasks can also not be compatible if they require significant crosstalk. That is, either using information from one task to perform another or when both tasks require similar mental resources. But tasks that are not mentally demanding and cause only minimal crosstalk can be performed simultaneously with much better results [7]. Therefore, throughout this thesis, the term *multitasking* can mean semi-frequent switching between high demanding tasks, or simultaneous engagement in multiple low demanding tasks. In the first case, if the tasks are granular enough, a person can switch to another task after finishing some small isolated step, such as reading and responding to one email after writing one simple unit test for their code. In the second case, we can assume that a person is far from fully occupied by any of the tasks individually. An example of such scenario can be walking a long distance in a video game on a straight road and messaging a friend. None of these tasks is critical, which means that potential spelling errors or walking off the road for a while can be tolerated, and the player is capable of paying enough attention to both tasks at once.

Keyboard and mouse

The first obvious existing solution for user input is the traditional combination of keyboard and mouse [2]. This can serve as a baseline for comparison with other solutions. It is a very versatile option and extremely spread. It has very low latency, on the order of single milliseconds, with some modern gaming hardware claiming latencies even below 1 ms. It is supported by virtually all operating systems and hardware platforms. While reliability can vary between different hardware models, it can be expected to be very high for multiple years of usage. Customization is often possible in the form of a wide range of different hardware models. After-purchase customization can be problematic on the physical level, and more and more hardware is now programmable to some extent, allowing users to change button mappings or sensitivity settings.

In terms of compactness, the keyboard and mouse combination stops being the ideal one. By design, it is not possible to use both efficiently at the same time. A user will often need to move hands between the two devices, which can be slow and interrupt the flow of work. This type of hardware is also problematic for multitasking between multiple

computers. Without additional software or hardware, the general solution is to have one set of keyboard and mouse per computer, multiplying the cost, space, and usage compactness issues. To allow multitasking between multiple computers with only a single set of keyboard and mouse, there are various switching solutions. Either hardware based KVM (Keyboard, Video, Mouse) switches or software based solutions that allow sharing keyboard and mouse over network connection. A closer look at these solutions is provided in Section 2.5. Such solutions offer an effective way to control multiple computers, but typically only one at a time. A use case such as programming client on one PC and server on second PC become significantly easier. But for example, writing notes while playing a game is not possible with those systems because the mouse and keyboard, or just their specific parts under specific conditions, are to be routed to different targets.

The second issue mentioned with the keyboard and mouse combination was the limitations with customization and compactness. Customization is often improved with vendor specific software that allows one to remap keys, create macros, or adjust sensitivity settings. However, these systems are tied to specific hardware, and portability can be limited, depending on the existence of internal memory on the device to store settings. Mapping options can also be limiting to some extent, as the rule of ‘one key, one action’ is often applied. The option for macro creation can improve this, but no unified standard for macro programming exists, making it difficult to transfer knowledge between different hardware or software systems. These programs can often limit the user to a specific set of actions, such as a simple sequence of key presses with optional delays in between. More complex actions, like conditional statements or loops, are often not supported.²

Other dedicated hardware alternatives do exist to improve compactness and other metrics, although this introduces additional cost and potential problems with portability and hardware continuity. Software solutions to improve these metrics, such as remapping keys, macros, or scripts, can be very effective, but they are often tied to specific operating systems or applications. Such transformations are typically done locally, and another solution needs to be added to allow multitasking between multiple computers or applications.

Configuring a system, where individual machines need to have very different configurations, can be a complex and time consuming task. Different operating systems, varying levels of hardware integration or virtualization, different languages or keyboard layouts, and more can all contribute to the complexity of such setup. However, such situations are not too frequent for average users. The simplicity of common setup might still outweigh any limitations that were discussed so far, especially if alternative solution is either costly or requires additional learning effort.

Alternative input methods

So far, only solutions based on the traditional keyboard and mouse combination were discussed. However, there are many alternative input methods that can be used to address the problems of fragmentation, customization, and compactness. The most common alternative is the touch input, which is widely used on smartphones and tablets. Touch input allows direct interaction with the screen, eliminating the need for separate input devices. This can improve compactness and ease of use, especially for casual tasks like browsing or media consumption. However, touch input may not offer the same level of precision or

²From personal experience, the last software supporting if or loop statements was Bloody4 from A4Tech. Since then, driver for Roccat Tyon had limited support, Razer Synapse, Logitech G-Hub or Azeroon Software do not support these at all.

efficiency for tasks that require extensive typing or fine control, such as programming or gaming.

Rather than providing the user with a hardware to directly control a system, some solutions convert observation of the user into input commands. Eye-tracking system can provide alternative to a pointing device by detecting where the user is looking on the screen. Using such system could completely eliminate the need for a mouse, allowing user to keep both hands on the keyboard. Current major limitations of eye-tracking systems are their latency, cost, and precision.

Motion sensors to detect hand, finger, or whole body movements can also be useful as alternative input methods. Optical systems using cameras for motion detection or inertial systems that use gyroscopes and accelerometers can detect user gestures and convert them into input commands. Such system is often used in virtual reality (VR) setups, where traditional input devices may not be practical. While motion-based input can provide a more immersive and intuitive experience, it may also introduce new issues and challenges related to accuracy, latency, and user fatigue.

Voice input is another great alternative method that has gained popularity with the advancement of speech recognition technology. Home voice assistants, speech-to-text applications, and voice-controlled devices are all examples of systems without the need for physical interaction. Hands-free operation can be useful for many scenarios, either due to physical limitations or out of convenience. Dynamic activities like cooking and exercising, or situations where touch input is problematic, such as driving or in sterile environments, can all benefit from voice input. However, noise, privacy concerns, and precision of speech recognition can be a limiting factors.

Another example can be found in the game industry, where specialized controllers such as flight sticks, racing wheels, or gamepads are used to provide more immersive and precise control for specific types of games. These controllers often feature unique button layouts and input mechanisms tailored to the gameplay experience. Although they can improve customization and compactness for gaming scenarios, they may not be suitable for general computing tasks and can require additional investment in hardware. In similar manner, attempts for wearable input devices exist, such as rings or wristbands with built-in buttons, touch-sensitive surfaces, or acceleration sensors. These devices aim to provide compact and convenient input options for specific use cases, such as controlling music playback or receiving notifications. However, their functionality is often limited compared to traditional input devices, and they may not be suitable for tasks requiring extensive input or precision.

2.2 Low-Level Input Processing

To fully understand the possibilities and limitations of user input systems, it is important to have a basic understanding of how input is processed at the low level within an operating system. This section provides an overview of the key concepts and components involved in low-level input processing. This section at first looks at the hardware side of how input devices detect and generate user input. Then it continues with various methods of how this input can be transmitted to the target device. Finally, it looks at how the operating system processes this input and makes it available to applications. Since these processes can be very different between various hardware devices or operating systems, this section focuses on general concepts that are widely applicable.

2.2.1 Input hardware - Generating signals

At the most basic level, input devices such as keyboards and mice consist of physical components that detect user actions and convert them into electrical signals. These signals are then transmitted to the computer for further processing. The specific mechanisms for detecting user input can vary widely depending on the type of device and its intended use. While deep study of hardware design and signal transmission is beyond the scope of this thesis, it is useful to have a basic understanding of how these devices work to see any potential limitations in their usage and why they are designed the way they are, which can inspire new solutions or identify areas for improvement.

Keyboard

A keyboard typically consists of a matrix of keys, each associated with a specific electrical circuit. When a key is pressed, it completes the circuit, allowing current to flow and generating a signal that corresponds to that key. The internal controller inside of a keyboard scans the matrix to detect which keys are pressed and encodes this information into a format suitable for transmission to the computer.

The detection of key presses can be implemented using various technologies, such as mechanical switches, membrane switches, or capacitive sensing. Mechanical keyboards use individual switches for each key, providing tactile feedback and durability. Membrane keyboards use a flexible membrane layer that registers key presses through pressure. Capacitive keyboards detect changes in capacitance when a key is pressed, allowing for more sensitive and responsive input experience. Among a few newer technologies are optical keyboards, which use light-based sensors to detect key presses, offering potentially faster response times and reduced wear and tear. Big potential benefit of optical keyboards is the possibility of analog keypress detection to provide controlled device not only the information whether a key is pressed or not, but also how much it is pressed. This can open up new possibilities for input methods in areas such as gaming or artistic applications.

The main output of a keyboard is a series of key codes, which are numerical representations of the keys pressed. These key codes are sent to the computer for further processing. Keyboards offer a simple and reliable input method for text entry, for which they are generally the fastest and most efficient option. For other types of input, such as pointing or gesture control, keyboards are less suitable. Analog keypress could improve this to some extent, for example, not only indicating direction of on-screen pointer movement but also its speed based on how much a key is pressed.

Although keyboards are well tested and established input devices, they do have some hardware limitations that most manufacturers need to take into account:

- **Key Rollover** — This refers to the number of keys that can be pressed simultaneously and still be registered correctly by the keyboard. Because a popular method for detecting key presses is scanning the keyboard matrix row by row, pressing multiple keys at once can lead to „ghosting“ or missed key presses. This is particularly important for gaming keyboards, where multiple key presses are common. Increasing key rollover capability results in more complex circuitry and higher cost.
- **Debouncing** — When a key is pressed or released, the mechanical contacts can bounce, causing multiple signals to be generated for a single key press. To prevent this, keyboards implement debouncing techniques, which involve ignoring subsequent

signals for a short period after the initial key press or release. This ensures that only a single signal is registered for each key action.

- **Feedback** — Keyboards can provide various types of feedback to the user, such as tactile feedback from mechanical switches or auditory feedback from key clicks. This feedback can improve the typing experience and help users confirm that their input has been registered. However, there is a trade-off, as stronger feedback mechanisms can lead to increased noise levels and potential user fatigue over extended typing sessions due to higher actuation forces required.

Mouse

A mouse typically consists of a sensor that detects movement and buttons that register clicks. The technology for buttons is similar to that of keyboards. While there are again many different technologies for movement detection, there are 2 main categories: mechanical and optical. Mechanical mice with the rolling ball work by detecting the rotation of the ball as it moves across a surface. This rotation is captured by internal sensors that capture the movement in X and Y directions as rectangular signals. Optical mice use an LED or laser to illuminate the surface beneath the mouse, and a sensor captures images of the surface at high speed. By analyzing changes in these images, the mouse can determine its movement in X and Y directions. Optical mice are generally more accurate and reliable than mechanical mice, as they are less susceptible to dirt and wear. Mechanical mice are now mostly obsolete, but the detection of mechanical movement is still used in other input devices, such as trackballs or scroll wheels, which are often integrated into modern mice.

Ignoring the buttons (which at this level do not differ from keyboard keys), the main output of a mouse is the relative movement in X and Y directions, typically represented as delta values. These values indicate how much the mouse has moved since the last report. Mice can also include additional features such as scroll wheels, which provide vertical scrolling input, and additional buttons that can be programmed for specific functions. They are very good for movement based input such as control camera orientation in 3D applications. To also fully control movement in 3D space, additional input is needed for the 3rd dimension. Mouse is now very often synonymous with pointer control, mostly as a long-standing tradition, but while they are good at this task, other devices such as touchscreens can be more suitable. That is due to the direct nature of touch input, allowing one to directly interact with on-screen elements rather than moving a pointer indirectly.

Touchscreens

Touchscreens are input devices that allow users to interact with a display by touching the screen directly. There are several technologies used in touchscreens. Resistive touchscreens consist of multiple layers that register touch when pressure is applied, causing the layers to make contact. Capacitive touchscreens use the electrical properties of the human body to detect touch. Infrared touchscreens use an array of infrared light beams to detect touch points when the beams are interrupted by a finger or stylus. Each technology has its own advantages and disadvantages in terms of sensitivity, durability, and cost.

Touch devices can have many intended uses. A normal touchscreen can be used as a direct and often considered more intuitive replacement for mouse input, allowing users to interact with on-screen elements by tapping, swiping, or pinching. A graphics tablet, on the other hand, is designed for precise input using a stylus, making it ideal for tasks such

as digital drawing. Graphics tablets usually provide pressure sensitivity for more nuanced control over brush strokes. A touchscreen usually cannot directly give pressure information, but this feature can be moved into the stylus itself, allowing it to communicate pressure data separately.

Touch input can provide a more natural and intuitive interaction method, especially for tasks that involve direct manipulation of on-screen elements. However, touchscreens may not offer the same level of precision or efficiency for tasks that require extensive typing or fine control, such as programming or gaming. This is well illustrated by the fact that touchscreens such as smartphones or tablets often use on-screen keyboards for text input rather than alternative text input methods, such as handwriting recognition. While latter is currently also often implemented natively, the default option tends to be an on-screen virtual keyboard. Inferior precision is also true for pointing tasks, where a mouse with low sensitivity will have higher precision than pointing with a finger on a touch screen.

2.3 High-Level Input Processing and Interpretation

Once the input signal is registered by the operating system, it needs to be interpreted to convey the user's intent rather than just some raw data. This section explores how operating systems or applications process and interpret input signals to provide meaningful interactions.

2.3.1 Outside of Anglosphere

Good example of the need for input processing above the basic low-level mapping might be found in countries outside of Anglosphere, where character outside of the ASCII set are used. The issue is most prominent in languages with large character sets, such as Chinese, Japanese, or Korean (often referred to with an acronym of *CJK*), where thousands of unique characters exist. Before the era of computers, mechanical index typewriters were used for the most common characters. They were organized in a grid, and the typist would select the desired character by moving a pointer to the appropriate position. While one could argue that this is not too far from modern touch screen virtual keyboards, memorizing the position of thousands of characters would be feasible only for professional writers.

Modern solution for CJK input is based on translating input sequences entered using a standard keyboard. The easier part of CJK to process are characters based on syllables, such as Japanese Kana or Korean Hangul, where each character represents a syllable rather than a single phoneme. These characters can be directly mapped from a specific input sequence. For example, Korean text is written by entering a sequence of consonants and vowels that are combined into a single Unicode character, representing the entire syllable. These characters are well placed in the Unicode standard to allow easy numerical calculation of the final character code based on individual components. This system allows simple input of syllabic symbols with a standard keyboard layout, but it isn't as straightforward method as direct mapping of keys to characters and multiple issues can arise. For example, the system does not know at the moment of the newest key press if it is last part of the currently constructed syllable or is it the first consonant of the next syllable. To solve this, the input method editor (IME) needs to implement some form of context awareness to know which syllable is being formed.

Even more complex is the input of logographic characters, such as Chinese Hanzi or Japanese Kanji, where each character represents a word or concept rather than a sound.

For this case, multiple input methods exist, each with its own approach to mapping input sequences to characters. One common method is the Pinyin system, which uses the Latin alphabet to represent the pronunciation of Chinese characters. Users type the Pinyin representation of a character, and the IME provides a list of possible characters that match the input. The user can then select the desired character from the list. This method requires the IME to have a comprehensive database of characters and their corresponding Pinyin representations, as well as context awareness to prioritize commonly used characters based on the input sequence.

Languages closer to the basic Latin alphabet, such as the Czech language with its diacritics, often uses a different approach. The so-called *dead keys* allow one to first input a diacritic mark without directly displaying it and then combining it with the next typed character. For example, to type the character 'á', a user would first press the dead key for acute accent ('), followed by the letter 'a'. IME then combines these inputs to produce the desired character.

2.3.2 Input Prediction and Correction

Especially on virtual keyboards, where the physical feedback is limited and the keys are often small, the potential for input errors can be high. To mitigate this and to improve the overall user experience, many input systems implement prediction and correction mechanisms. These mechanisms analyze the input sequence and provide suggestions or automatically correct errors based on context and user behavior.

The other side of the coin is input completion, where the system predicts the intended input based on partial input. Very successful method for this task was the *T9* (Text on 9 keys) system. A technology for predictive text that aims to eliminate need for extra keystrokes. Each of the 9 keys on a traditional mobile phone keypad is assigned a group of letters, which is selected by multi-tapping the key. With T9, user selects just the group with single key press, and the system finds possible words that match the input sequence, selecting the most frequent one by default.

With improvements in hardware performance and machine learning algorithms, modern input systems can implement more advanced prediction and correction techniques. These systems can analyze user behavior, context, and learn from the user for customized suggestions. Due to this, they should be able to provide more accurate predictions than simple dictionary-based approaches. Such systems are now widely used with virtual keyboards on smartphones and tablets.

With the recent advancements in Large Language Models (LLMs) in text processing and generation, new techniques for input prediction are emerging. A great example of this is the GitHub Copilot [32], a code completion tool that a user can use to generate code snippets based on context. One of its features is the ability to suggest lines or blocks of code as the user types, effectively doing text prediction for programming tasks. This has the potential to significantly speed up entering repetitive or boilerplate code, allowing the user to focus on higher-level logic and design. While it is not directly an input method, it is a great example of how advanced prediction techniques can enhance user input experience.

On the other hand, for more complex programming tasks, over-reliance on such predictive systems can lead to multiple issues. Only time will tell whether these systems can reach a sufficient level of accuracy and reliability to be widely adopted and are currently only in the early stages of development or if they continue to be usable only to automate simple tasks. As for GitHub Copilot, the more complex the suggested code needs to be, the more

likely it is to contain errors or not follow best practices [20]. In addition to errors and potential security risks in the generated code, there is also a high risk of reduced learning abilities, shortened attention span, diminished critical thinking, over-reliance on AI, and outright reduction of critical thinking and cognitive skills [9, 31].

2.3.3 Pointing Device Gestures

Another form of high-level input processing is the detection and interpretation of gestures. Those are specific movements or patterns made with a pointing device, such as a mouse or touchscreen, that convey specific commands or actions. Using motion gestures can provide an intuitive and efficient way to interact with applications, offering either a simpler replacement for keyboard shortcuts or complex menu navigation, or providing a convenient alternative for a simple actions [1]. Gestures can be simple, such as a swipe or tap, or more complex, involving multiple movements or combinations of inputs. Full replacement of keyboard as a whole might be possible when combined with other techniques [10], but it is often more practical to use gestures as a complementary input method alongside traditional keyboard and mouse input.

The intuitiveness of gestures can be their biggest advantage. Mimicking real-world actions, such as *stretching* an image to zoom in, can make it very easy for users to understand and remember. However, the inherent ambiguity and imprecision of gestures can cause challenges in accurate recognition and interpretation, while also limiting the available input vocabulary. The question of how to even inform users about available gestures and how to perform them correctly is also non-trivial.

2.4 Ergonomics and Human Factors

It is important to consider human characteristics when designing input systems. The comfort, efficiency, and overall experience of a user can be significantly impacted by ergonomic factors. Prolonged use of either poorly designed hardware or improper user habits can lead to discomfort or even injuries [6]. Factors such as device shape, button placement, hand movement, or required keypress force can all contribute to user fatigue or strain. Some factors can be improved by input customization via software, such as remapping keys to reduce the travel distance of a finger or creating macros to automate repetitive tasks. Other factors require careful hardware design, such as ergonomic keyboards that promote a more natural hand position, such as split keyboards [13] or adjustable keyboard angles.

Another part of human factors is the cognitive load. Input systems should aim to minimize the mental effort that is needed to use them effectively. While this is mostly in the field of user interface design, input methods can also contribute. Complex gestures, multiple layers of key mappings, or inconsistent input behavior can all increase cognitive load and make it more difficult for users to interact with the system efficiently. As an example of such increase in cognitive load can be found in the modifier keys. While they provide additional input options without increasing the number of physical keys, they also require users to remember additional key combinations. Especially actions that change the *mode* of input, such as switching between different keyboard layouts or various layers such as *Function Lock* or *Caps Lock*, can lead to confusion and errors as user needs to constantly keep track of the current input mode. In his book *The Humane Interface*, Jef Raskin proposes the idea of *modeless* input, where the system avoids changing input modes at all. This can

be replaced by *quasimode*, where the mode is only active while a constant physical action is provided, such as holding down the Shift key instead of toggling Caps Lock on or off [24].

Responsiveness and format of the provided feedback is also important for the user experience. Significant delay between the user action and any conformation of receiving such action can disrupt the flow of work and lead to inefficiency and discomfort. Feedback from the hardware itself, such as change in resistance of mechanical keys, audible clicks, or a vibration, can provide the user immediate confirmation of their input. Similar confirmation needs to also be provided by the software, to notify the user that their input was really received and processed. When typing text or other direct actions, this can be as simple as updating the display to show the entered character. But for example, when changing modes, using a dead key, or performing complex gesture, it can be useful to feedback also for any in-between steps, to confirm that the system is in the expected state. Such confirmation can include popup icons, change of cursor shape or color, or auditory feedback.

It should also be noted that different users can have different preferences and some users can be more sensitive to certain forms of feedback than others. Input systems should ideally provide options for customization, allowing users to tailor the feedback mechanisms to their individual needs and preferences. Vibration feedback can be a good example, as it can be very helpful and distinctive, but it can also cause discomfort for some users.

2.5 Existing Systems

The issue of fragmented input across both single and multiple devices is not new. Many existing systems and solutions have been developed to address various aspects of this problem. It is worth taking a look at some of these systems to understand their approaches, strengths, and limitations. This section provides an overview of selected existing systems that aim to improve input handling and multitasking capabilities.

2.5.1 AutoHotkey

Designed for Microsoft Windows, AutoHotKey (AHK) is a powerful and versatile scripting language and interpreter that allows users to create custom scripts for remapping keys, hotstrings, or macros [12]. AHK scripts offer a great degree of customization, allowing users to implement both simple and complex input transformations. It can also continue to run in the background to perform various long-running tasks, such as monitoring input or system status or continuous input emulation.

AHK is widely used to automate repetitive tasks, create custom keyboard shortcuts, and improve productivity [4]. Thanks to its flexibility, it can be adopted for an impressive number of use cases. However, some limitations exist:

- **Platform Dependence** — AHK is being developed specifically for Windows operating system. While there are some alternatives for other platforms, such as AutoKey for Linux, AHK itself is not natively supported outside of Windows.
- **Requires Scripting Knowledge** — While AHK is relatively easy to start with, reliance on text-based scripting can be a barrier for users without any programming experience. Implementing more complex behavior requires a good understanding of the AHK scripting language and programming concepts in general.
- **Security** — As any other scripting system, AHK scripts can be tempered with or planted with malicious code. Executing scripts from untrusted sources can pose huge

security risk as AHK has access to all system resources either directly, via Windows API, or in the most paranoid case, via NumGet and NumPut functions to read and write arbitrary memory locations. Best practice is to use compiled AHK scripts (.exe) for distribution, as this should be as secure as any other compiled program. Compiling scripts although hides the option of modifications, so users can not further customize them. And user still needs to trust the original author of the script.

- **Performance Limitations** — As an interpreted scripting language, AHK may not be suitable for performance-critical applications. Scripts that require real-time responsiveness may face limitations due to the overhead of the interpreter.³
- **Anti-cheating Measures** — Competitive videogames often implement anti-cheating measures that can flag AHK execution as potential cheating attempt. This can lead to bans or other penalties for users employing AHK scripts, even if they are not used for cheating purposes.⁴ Some games can cause technical issues when combined with AHK, such as incorrect input handling or resetting input hooks and thus breaking the AHK scripts.
- **Limited Network Capabilities** — Networking support is not natively built into AHK. This can be overcome by either libraries or using Windows API calls, but this add complexity and can be a barrier for implementing networked input solutions.

2.5.2 Serenade

Although no longer actively developed, Serenade can still serve as an interesting case study of a voice-controlled input system [28]. The aim of Serenade was to provide hands-free coding capabilities, allowing users to dictate code using voice alone. It supports multiple programming languages and strives to provide full IDE functionality through voice commands.

This system, or any other way of controlling the computer by voice, can be very useful for users with physical disabilities that either prevent them from using traditional input devices or make their use difficult. It can also provide an alternative input method for situations where hands-free operation is desired, such as when multitasking or when a physical input devices are not available. But it can also be used by general users as an alternative form of input. Some of the reasons for this can include reducing arm and hand strain, multitasking while performing other activities, personal preference or bad access to traditional input devices. As an example, searching for a keyboard and mouse while using a virtual reality headset can be difficult, while voice input can provide a convenient alternative.

Some of the limitations of Serenade might be:

- **Accuracy and Reliability** — Voice recognition technology has improved significantly in recent years, but it can still struggle with accents, background noise, or complex vocabulary such as what is needed for programming. This can lead to frustration and reduced productivity if the system frequently misinterprets commands.

³It is worth noting that simple hotkeys and hotstring do not effect system performance in any noticeable way. Executing actual scripts can introduce minor delay, but for most use cases this is negligible. For competitive gaming or other scenarios where every millisecond counts, this still can become an issue.

⁴From personal experience, using AHK for hotkeys and hotstrings for many years never caused any issues in games. For competitive games I follow the rule of *One keypress - One action*. For single-player games with quick-time events or similar mechanics, using AHK can be very helpful. However, some publishers such as 2K Games have policies that ban such actions even in single-player games.

- **Active Window Only** - Serenade operates as any other form of input device, meaning it can only send input to the currently active window. This limits its multitasking capabilities, as users cannot control multiple applications simultaneously.
- **Security** — As with any other external tool, it can pose security risks. Running everything over localhost mitigates some of these risks, but users still need to trust the software with potentially sensitive information.
- **Privacy** — Using voice can be disruptive in public environments. Additionally, concerns about data privacy arise with the possibility of unwanted recording or listening.
- **Responsiveness** — Current voice recognition systems often introduce noticeable latency before executing issued commands.

2.5.3 Remote Desktop Software

As for multitasking across multiple systems, remote desktop software provides a way to control multiple computers from a single input source. Many products exist in this category, such as Microsoft Remote Desktop, TeamViewer, or VNC-based solutions. The intended use case is to allow users to fully access and control a remote computer as if they were physically present at that machine, transmitting input from the local device to the remote system and displaying the remote system's output on the local device.

Great advantage of remote desktop software is the seamless usage of multiple systems on local machine. In full-screen mode, other than latency caused by network transmission, it can be indistinguishable from using local system. Many products also support multiple operating systems, allowing cross-platform multitasking. However, some limitations exist:

- **Security** — As with any other software that uses network communication, both host and client need to trust the software, network connection and each other.
- **Latency** — Latency can be significant issue, where even for simple tasks like text input or mouse movement, delay can disrupt user experience.
- **Lack of Transformation** — Input is typically sent as-is, without any transformation or customization options. This can be especially limiting on multi-language systems.
- **Resource Intensive** — While for truly remote sessions (that is accessing a computer that is physically distant) this is unavoidable, using remote desktop software for local multitasking can result in unnecessary resource consumption.⁵
- **Active Window Only** — Many products strive for better integration, for example, by automatically capturing the mouse when hovering over the remote desktop window without it being active, it still fundamentally operates on the active window principle. Same as with voice control systems, this limits multitasking capabilities. For some products, the limitations is set even further, as only single user can control the remote system at a time, including the local user physically present at the remote machine.

⁵Good example is using remote desktop to access personal PC from a laptop in the same room. Using both systems directly would lead to big fragmentation. Using just the PC might cause issues with accessing files and applications on the laptop. Using remote desktop software to access the PC from the laptop can solve this, but even over LAN this introduces noticeable delay compared to selectively sending input events from the laptop to the PC and rendering PC output on dedicated monitor connected to the PC.

2.5.4 KVM switches

A good alternative to remote desktop software for local usage is a physical KVM (Keyboard, Video, Mouse) switch. It is a hardware device that allows users to control multiple computers using a single keyboard, monitor, and mouse. By pressing a button on the KVM switch, users can switch control between connected computers. This can be a great solution for setups with multiple physical machines, such as a workstation and a laptop. Due to a direct hardware connection, KVM switches typically have very low latency and provide reliable input transmission. Transmitting video output from multiple systems to a single monitor offers seamless switching between systems without the need for network communication. However, some limitations also exist:

- **Cost** — Since KVM switches are physical devices, they can be relatively expensive, especially for models that support multiple computers or high-resolution video output.
- **Limited Customization** — KVM switches typically do not offer transformation of the input or customization options.
- **Physical Constraints** — KVM switches require some physical connection between the switch and each computer. This limits number of possible connections, requires good cable management, and further constrains the physical layout of the workspace.
- **Limited Multitasking** — Input and output are tied to one active system at a time. Although switching is quick, users cannot control multiple systems simultaneously.
- **Laptop Support** — Having a laptop as one of the connected machines might not be a problem, if the correct ports are available. But because keyboard and display are often integrated into the laptop itself, KVM switch cannot utilize them, and laptop cannot be thus used as the input device. An external keyboard and monitor is still needed, which can reduce the overall convenience of using a laptop.
- **Output Limitations** — Full potential of GPU capabilities might be underutilized as KVM switches often have limited number of ports, and might not offer support for high refresh rates or advanced features such as HDR.

2.5.5 Synergy

As presented in section above, using physical KVM switch might present multiple issues. Software-based variants such as a Synergy can be a good hybrid solution that combines some advantages of a remote desktop software and KVM switches [27]. In a similar way to remote desktop software, Synergy listens for a user input on one local machine and transmits it over the network to other connected machines. But instead of working in a local window where the remote desktop is displayed, the target system's display is considered to be just an extension of the local machine, effectively acting just as an additional source of user input for the target machine.

Compared to a remote desktop software, this approach offers a seamless experience, reduces latency, and consumes less resources. Higher level of multitasking is also possible, as no physical action for context switch is needed and multiple monitors connected to different systems can be used simultaneously. The biggest downside is, with respect to this thesis, the lack of input transformation and selective redirection options. Looking at the example presented at the beginning of this chapter, transmitting only a few selected keys while keeping the rest of the input local is not possible with Synergy.

Chapter 3

Motivation and Design

As it was discussed in a previous chapter, the main goal is to design a system that would allow easy multitasking by controlling multiple systems that can be configured to a wide range of user needs. This chapter discusses the overall design of the system, individual components, and their interactions. First, the Section 3.1 discusses a potential use cases that are aimed to be covered and provides the basic concepts that the designed system should follow. Section 3.2 focuses on the transformation system, from detecting user input to creating an internal structure representing the processed input. Section 3.3 discusses a potential routing of the input events to various targets. The section 3.4 looks into configuration options and control of data flow inside the system.

As was shown in Section 2.5, currently existing systems do provide good options for these functionalities, but are often limited in their configurability, extensibility or portability. Therefore, the designed system is not supposed to be one final product that satisfy all user needs, but rather a framework that can be easily adapted into other systems and applications or expanded with new functionalities as needed. It should also provide a clear communication protocol so that other independent software can be developed without reliance on this framework and still be able to interact with it. To avoid any confusion, throughout the rest of this thesis, the term *system* or temporary name *InputResender* will be used to refer to the designed framework. For a ‘system’ as a computer setup with peripherals and other components, the term *machine* will be used. For the operating system, either *OS* or the full name will be used.

3.1 Motivation

The example provided at the beginning of Chapter 2 illustrated some use cases for a system that is able to selectively transform user input and potentially route it to another application or device. This will serve as the main motivation for the design of the system. But other use cases can also be considered. While the main idea of this system is to provide given functionalities under one unified framework, it can be split into these two main parts: input transformation and input routing.

The first part, input transformation, is focused on accepting user input from various devices and processing based on user defined rules. Various use cases for this part are shown in Figure 3.1. Support of different layouts here is not only limited to keyboard layouts, as is presented by operating systems, but rather can be seen as mapping of any input sequence, state, and context into another input sequence, state, and context. The example with

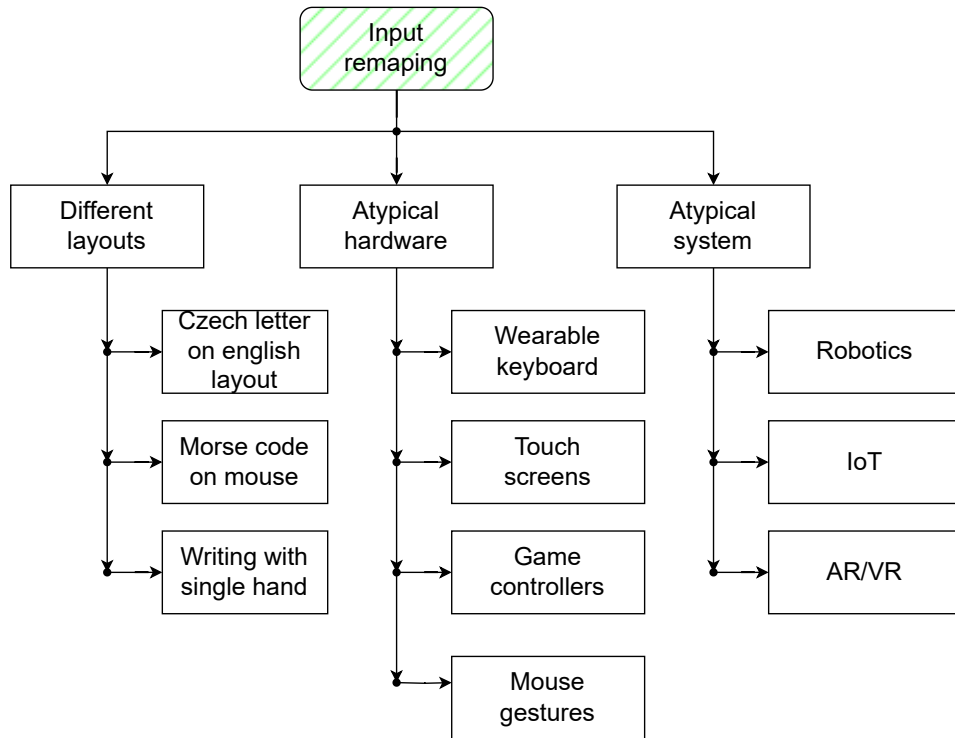


Figure 3.1: Diagram showing various use cases for the transformation part of the system.

writing in Morse code, as mentioned in previous chapter, is one such case. Other can be writing using only one hand, using just 5 keys to write the entire alphabet by emulating the functionality of a wearable keyboard [21].

Even though not all use cases are to be implemented as part of this thesis, it is important to keep them in mind during the design so that the system can be easily extended to cover them in the future. One part of the system is to allow experienced users to expand the system themselves for their own needs. This is to be common concept throughout the design of the system. For this, we can define an *expert-user* as a user who has an advanced programming background, is knowledgeable about this system, as well as about the target operating system, hardware and applications, and is able to estimate potential security and stability risks that can arise from using this system. To avoid unnecessary initial skill requirements for new users, the system should also provide an easy to use user interface for basic configuration and control, preferably with graphical elements.

Such tasks could be achieved by various existing software, such as AutoHotKey, as discussed in Section 2.5. However, a lack of portability options and a simple user interface can make it difficult to implement. More advanced tasks, such as gesture detection, would become even more difficult and in the case of AHK, would probably be implemented in an external library in a similar way as is intended for this system. Even a complex scenario, like controlling a drone using some MIDI instrument, could probably be achieved in AHK or similar scripting software via external libraries and system calls. But portability would remain a major issue.

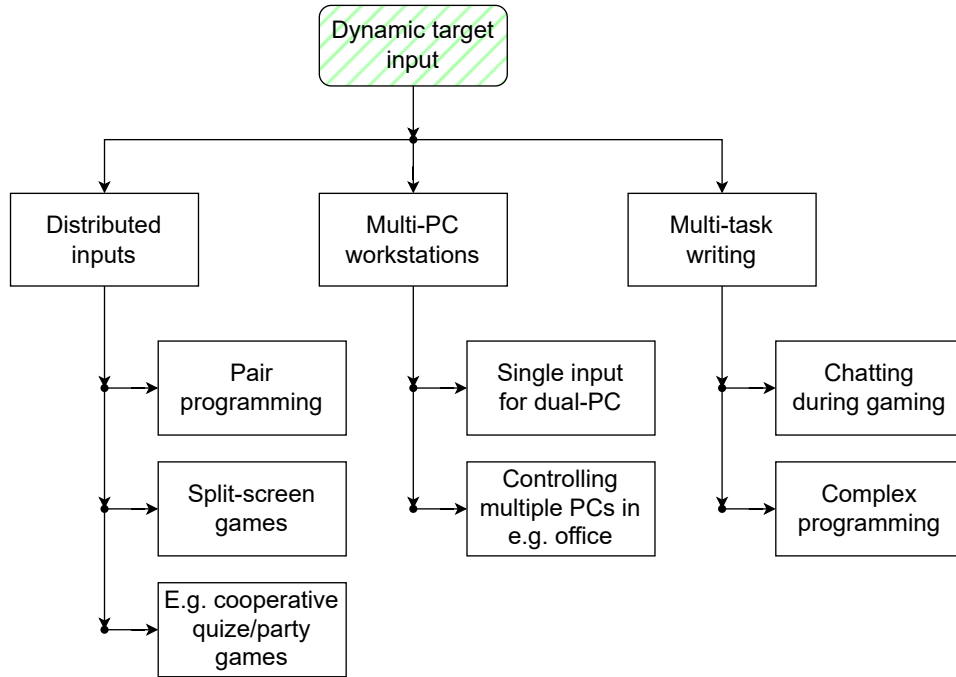


Figure 3.2: Diagram showing various use cases for the transformation part of the system.

The second part, input routing, is focused on sending the transformed input to various targets, as shown in Figure 3.2. This can be a 1:1 communication, where an input is simply forwarded to another device. Under ideal conditions, operating system would redirect an input to another machine or a specific application running on the same machine as easily as it processes real input from connected peripheral devices. However, this is highly dependent on the targeted application and operating system capabilities. Sending input directly to a specific application is often considered a security risk and is often simply ignored either by the application or the OS itself. This is why it is also desired to provide a protocol for communication, so that applications can implement support for receiving input from this system directly, if needed.

Selective 1:N routing is another use case that is to be supported. The basic idea offered by physical or virtual KVM switches is the basic variant of this use case. That is, a user can select which machine is currently active and all input is forwarded to that machine. A more complex scenario is a conditional routing, where a target is selected based on some criteria, either the content of the input event itself, or some external context. As InputResender is not supposed to be just for mapping input, new virtual input events can be generated during processing and routed to different targets.

The place of this system lies in the area between these two parts. Combining both parts together, the system should offer a higher level of configurability, portability, and performance than combining multiple existing solutions together. Thanks to modular design, it should provide a highly customizable solution that does not directly replace existing software, but complements them, offers an alternative for new use cases, and delegate tasks to other software where appropriate.

There are several important concepts, aspects, and requirements that must be considered during the design of the system. Those are as follows:

- **Security** — User input often contains very sensitive information, such as passwords or intellectual property. Therefore, it is important to limit any potential security risks that can arise from using this system. Simultaneously, it is undesirable to limit the system capabilities and introduce limitations that would prevent certain use cases. The system should be limiting by default, but allow expert-users to disable such limitations. In such a case, enough information should be provided so that the user can assess the current status and potential risks.
- **Portability** — The system should be designed in such a way that it can be easily ported to different operating systems and hardware platforms. This is important not only for a wider adoption, but also to future-proof the system. Implementing native support is preferable, with Windows and Linux being the main targets.
- **Extensibility** — Providing an easy way to include custom functionalities is also important. Many use cases can be very specific, and it is impossible to cover all of them out-of-the-box. Therefore, the system should provide clear interfaces and documentation for creating custom modules that can be easily integrated into the system.
- **Selectiveness** — The ability of the system to decide when to apply certain transformations or routing is the main requirement for this system. Therefore, context should be accessible throughout the entire system and from all native or custom modules.
- **Performance** — Noticeable input delay can downgrade user experience and may outweigh any potential benefits that the system can provide. Therefore, it is important to keep the latency of both transformation and routing at reasonable levels.

It is now worth taking a look at the overall architecture of the designed system, before diving into individual components. A modular plugin-like architecture is proposed. The system is to be composed of seven layers, each using only the layer directly below it, and providing functionalities to the layer directly above it (with the exception of services that can be accessed from any other layer). An object on the same layer should be as separated as possible. Communication between them should be done only by objects on a higher levels [15]. The layers are as follows (from bottom to top):

- **Services** — Independent services providing various simple functionalities.
- **Core** — A layer that contains all registered components of the system and manages their lifecycle. Provides essential objects for depending layers to derive from, such as base component class.
- **Definitions** — Interfaces and data structures without any implementation details. Each definition is accompanied by the most basic and platform agnostic implementation and a set of general test cases, to enforce the expected behavior of any component that is deriving from this interface.
- **Implementations** — Specific implementations of various definitions and core object. Dependency should be kept to minimum.
- **Factories** — Module creation and management.

- **Application** — Highest layer that is still independent of both hardware and operating system. Very close to the 'Control' part in MVC architecture.
- **Platform** — Hardware or operating system specific implementations. Also serves as a top-level layer and entry point into the system. It can be separated into multiple layers or other architecture outside of this system.

3.2 Input Transformation

For any input transformation system to work, it first needs to be able to obtain some input signals. Because one of the requirements for the system is to be independent of both hardware and operating system, it will be assumed that there is a set of methods working on lower level of this system. Details about specific implementations of such methods are discussed in Section 4.1. Access to these methods is provided by one unifying component, *InputReader*. The required functionalities provided by these methods are the following:

- Register to receive information about input events.
- Generate custom input events.
- Unregister from receiving input events.

Because the system needs to allow a wide range of transformations, it is necessary to keep the context that surrounds any registered event. While a simple option could be to allow checking for the current state of any given input device, for example by testing if a certain key is currently pressed, this could interfere with the idea of selectiveness and also potentially introduce dependencies on specific hardware or operating system. For such a case, higher levels in the system would need to be aware of the type of hardware that is being used. The more general approach is to dynamically keep track of what events have occurred in the past and use that information to determine the current state of any input device. This way, the system can remain independent of both hardware and operating system, while still providing the necessary context for transformations. At the moment of writing, the proposed structure for an input events is as follows:

- **Device ID** — Unique identifier of the input device that generated the event.
- **Event ID** — Unique identifier of the event within the device.
- **Timestamp** — Time when the event occurred.
- **Event Type Mask** — Bitmask representing the type of the event, such as a key press or a mouse move.
- **Input Code** — Hardware specific code of the input event, such as a scan code for the keyboard key or even a virtual command ID.
- **New Value** — 3D vector representing the new value of the input event. For digital inputs such as a keyboard, simple 1 or 0 would be stored in the first element of the vector. Using a 3D vector allows it to represent various types of inputs, such as mouse movement, joystick position, steering wheel rotation, et cetera.
- **Delta Value** — 3D vector representing the change in value since the last event.

The amount of data that would need to be stored and passed around to inform about the entire history of all input events would be very impractical. Instead, it is proposed to keep only events since the last change of an input direction, by which is meant the last time when any of the delta value elements changed sign, combined with the state when the change occurred. This should provide enough context while keeping the amount of data manageable. This is considered as *low-level input context*. This context is constructed by *InputParser* component from single input events received from *InputReader*.

Following this, higher level processing can begin. This is to be handled by the *InputProcessor* component, the final part of the integrated transformation pipeline. the most generic view at this component is that it accepts low-level input context and executes some processing, based on specific implementation. It is recommended to produce boolean output, which indicates whether the input event is to be consumed by hook or passed on to other listeners in the hook chain of the operating system. However, when the *InputProcessor* is connected to other components in the pipeline, they might expect different output. But to prevent uncertainty about the output format, it is recommended to either stop processing pipeline at the *InputResender*, or return the boolean output back to the starting hook, while any other dependent components are notified by separate pipeline call.

The main consideration for any processing is the delay sensitivity. There can be a limit imposed by the operating system on how long an input event processing can take [16]. Depending on implementation, this can cause anything from simply ignoring consumption of the event and passing it on to another registered listener, to unregistering the input hook entirely, causing loss of input until re-registered. To address this, processing should be split into two categories: *Real-time*, which on one hand should be done as fast as possible, but on the other hand is guaranteed to finish safely within any given time limit, and *Delayed*, which will be run in separate thread and can take as long as needed, in which case concurrency issues and order of events are to be defined by specific implementation, by default should run in thread-safe manner and order of events to be random but non-breaking. To allow faster response for complex tasks such as gesture recognition, real-time mode can accept data, process them independently in a separate thread, and once results are ready, integrate them back into the real-time processing pipeline. This however requires careful design of such modules to avoid introducing noticeable delays, especially if synchronization is needed.

3.3 Input Routing

After processing an input event, it must be routed to an appropriate target. For simplicity, each target must be separate and any input event should be routed to one single target per action. If multiple targets are needed, multiple routing actions should be requested. This will be done while processing the input event, so that routing is part of the transformation pipeline. Therefore, anything outside of the input processing pipeline can therefore be completely oblivious to what data are inside the requested action.

Virtual Input

For many use cases, it is desirable that the processed input event appear as if it were generated by a physical input device. For example remapping of a keyboard keys should ideally appear as if the user is pressing different keys on the same physical keyboard.¹ For

¹The intention of acting as if the input is coming from the same physical device is not to actually spoof the device identity, but rather to avoid issues with how applications handle inputs. Some inputs can be

this purpose, the input processing pipeline should be bi-directional. That is, processed input events can be sent back to the *InputReader*, which will pass any given data to the lower level method to process the event. Due to the limitations in operating systems, it is not guaranteed to be indistinguishable from a physical device input, but the component that can capture input events should be the best candidate for such task.

Direct Input

For some situations where direct input into a specific application (specified by process ID or window handle) is possible, it should be provided as an option. If *InputResender* wants to be able to send an input directly to a specific application, it is only fair that it is able to receive targeted input as well. Under this condition, it does not differ from listening for low-level input events and can be handled by the same pipeline as a virtual input. As this scenario is often considered a security risk, it should be as an opt-in functionality, with very specific configuration options to avoid unintentional misuse. At the time of writing, this functionality is not implemented in the prototype and is left for future work. Under a Windows OS, this functionality could be achieved with the *SendMessage* call. With other operating systems, this can present a greater challenge. Although the most important aspect is ensuring the security of the received messages.

Network communication

The generic solution for routing input events to any target is to use network communication. Any target that is able to receive data over the network can be targeted this way. Applications running on the same machine can be reached via a loopback interface, remote targets can be reached via a local network or the internet. This approach has some drawbacks. In addition to a latency and security concerns, two important issues need to be addressed. First, it is the existence of firewalls and NAT devices that can render direct communication problematic. Solving this issue with techniques such as NAT hole punching is outside of the scope of this thesis, but it could be added by a custom implementation of an external module. The second issue is that the target application needs to be already running and listening on a specific port when the input event is being sent. This can be solved, for example, simply by starting the target application (whether it is another instance of *InputResender* or some other software) automatically with a startup of the operating system.

3.4 Configuration and Control

Since configurability is one of the main goals of this system, all the components and functionalities discussed previously should be easily configured and replaced if needed. They should provide only the baseline, not the only solution for any given part of the system. In addition to various specific configuration options for individual components, there are 4 main ways in which the user can control the system.

outright ignored if they are not generated by supported type of a device. For example, video games often ignore keyboard and mouse input if a controller is connected and vice versa. Other applications consider virtual devices as a security risk and ignore them.

Console Interface

The entry-point into the system is the console interface. This allows for including the system into various scripts and automation tools or to be controlled remotely via SSH or similar protocols. The system itself is to operate a virtual console to allow other forms of user interface to interact with it without relying on specific implementation details. This virtual console should provide only basic commands to control the system, such as listing / loading / unloading of modules and other commands. Other commands for individual modules should be provided by the modules themselves, registered into the console on demand, probably together with loading of the module itself.

To allow unified control over components of the system, it is suggested that the commands are bound to specific methods in the module code and that the console command is simply a front-end for calling these methods with given parameters. In this way, modules can communicate together as if they were executing commands from the console, without the need to create text-based commands and parse them back into method calls. This system is not currently implemented in the prototype, and the effectiveness of this approach remains an open question, as calling commands even between modules should be rare enough to not introduce noticeable overhead even when using text-based commands, which is supported by the general performance testing in Section 5.1.

External Libraries

As mentioned above, InputResender is intended to function primarily as a framework. Not only can other software incorporate it as a library to build on top of it, but InputResender itself should be able to load external libraries to expand its functionalities. Although not currently implemented in the prototype, it is intended that the user can provide a path to an external library via a console command, which will be dynamically loaded into the system and an entry point called to give control to the library. The library can then register additional components, commands, or replace existing ones as needed.

The biggest downside of this approach are potential security risks, as such libraries will have full access to the system. Therefore, not only should this functionality be protected from unintentional misuse, but the component responsible for loading libraries should also provide enough information about the loaded library to verify its authenticity and safety, such as digital signature and hash of the library file.

Pipelining

Because it was specified that individual modules should not communicate directly with each other and since modules can be replaced on demand, it is necessary to provide a way to connect various modules together on a higher level. This is to be done by *pipelining*. Individual components, such as *InputParser* and *InputProcessor*, provide information about what types of data objects they can understand. The user can then select a sequence of components that will be connected together into a pipeline with a single command. Once a component generates some output, it is offered to registered components that can accept such data, one by one, until one component accepts the data. This way, specific behavior can be achieved without the need to hardcode connections neither in the individual modules, the core system, nor in the higher level application. The user is also free to either use a generic definition of the component, a specific implementation, or even an explicit name (if such exists) to select which component to use in the pipeline.

Scripting

To allow introducing complex logic into the system without the need for custom compiled modules, scripting support should be provided. In a similar way as console commands, the semantics of the scripting interface is to be provided by individual modules, rather than by the system or the scripting engine. The major motivation for this decision is to provide a secure alternative to add custom logic into the system without providing full access to the system, as is the case with external libraries. Script code will contain a list of modules it wants to use and no other functionality can be accessed. This list, including the version number and hash of the module files, will be verified against the allow / block lists and only then is presented to the user for confirmation before loading. In this way, the user clearly knows what functionalities are requested, can inspect the code if needed, and then decide whether to allow it or not.

Because dynamic interpretation of script code can be very slow, scripts are pre-compiled into execution-friendly data structures when loaded. This first serves as a check for any syntax errors or forbidden operations and, secondly, significantly speeds up execution. Performance is estimated in Section 5.1. To further improve performance and provide additional startup flexibility, the concept of directives is introduced. They can execute system commands during the compilation phase, such as loading external files, constructing data structures, announce expected input or output variables, or calling console commands. In this way, any complex operation can be pre-computed during a compilation phase, and only final results are executed during runtime.

Individual controls and configuration options should allow for any of the other methods to be included as needed. For example, a user can create a script that calls several console commands in a configuration phase, either via a string commands or a direct parsed method calls, then creates a pipeline of modules and provides data to be processed by this pipeline. The pipeline itself can request execution of some script, for example, when processing an input event, and let the script decide where to route the resulting event.

3.5 Security

As mentioned above, security is one of the main concerns of this system. But the term security can be very broad and can be related to various practices and techniques. A secure system in general is important not only to protect sensitive user data and to not compromise stability of the machine, but also to gain user trust and wider adoption. Therefore, it is important to consider security from the very beginning of the design process and to implement various measures to mitigate potential risks. For the purpose of this thesis, security aspects can be divided into three main categories.

Error prevention

Any change in data or behavior of the system, which is not intended by the user, can be treated as a form of an error. Such errors can arise from various sources, such as hackers, bugs in the code, bit flips in memory, hard drive corruption, data race conditions, etc. The actual source of the error is not relevant. For example, from the point of view of a user, it does not make any difference if forwarding input events to a compromised target is caused by a malicious attack or by a corrupted writing into a configuration file.

There are many techniques to prevent this type of error, such as error correction codes, checksums, digital signatures, data duplication, et cetera [14]. For simplicity, it is assumed that the code does not contain any bugs, as fixing errors in the codebase is an essential part of the development process itself and is not specific to this system. It is also worth noting that under this definition, even malicious alteration of some data is considered here as an error. But in scenario, where the user is informed of such alteration and decides to allow it, it no longer fits the definition of *error* and becomes a form of *informed user decision*, which is discussed in the next section.

Data problems can be detected by using a hash function to generate a unique fingerprint of the data and then comparing it with the expected value. With the assumption of completely correct code, errors can arise from external sources or from hardware issues. Preventing hardware from causing errors is considered to be out of the scope of this thesis.²

Therefore, the focus is on preventing errors that are present in external data sources. This can be any file located on the machine, such as configuration files, script files, external libraries, et cetera, or any data packets received over the network. Data structures passed by the operating system, other software, or even by other components of this system could also be potentially corrupted, but options to verify such data are often limited and therefore are not considered here.

A common issue with validating data with hash functions is the question of where to store the expected hash values. It is not realistic to expect the user to manually enter the hash values every time from their memory. Any location where expected hash values are stored can be potentially compromised, and thus would require to also be protected. The proposed solution is to automatically construct a dynamic allow list for specific data sources. An expert user can manually add an expected hash for specific file path, but the main method is by automatically called commands that add pre-calculated hash values for specific files to the allow list.

These automatic commands are stored in a configuration files, and might be executed during startup of the system or on demand by a user. Because configuration files can be easily altered, they are also not considered as a secure source. To overcome this, configuration files are to be protected by a password. More specifically, a hash value is calculated from the content of the configuration file and the hash is encrypted with a user-provided password and stored inside the configuration file itself. When a configuration file is loaded, a user is prompted to enter a password, which is then used to decrypt the hash value and compare it with the hash calculated from the content of the configuration file.

If the hash values of either some loaded file or the configuration file do not match, it is proposed that a non-expert user is simply notified about the issue and that the file will not be loaded, while an expert user is notified about content of the file and asked for confirmation, which updates the stored expected hash value for the file. This way, expert users are not required to manually calculate and enter hash values if they understand what changes were made and are confident that they are not malicious, while non-expert users are protected from potential risks without overwhelming them with technical details.

²The first step in preventing hardware errors is deciding how strong prevention is enough. For a common user, there might not be any need for such prevention at all. For business uses, for example, investing in a hardware with built-in error correction and duplicating critical data can be worth the cost. But there is no specific line that can be drawn here, as a hypothetical argument can be made, that this software could be used in a space craft sent to Mars. In such a case, how complex error prevention system can be, is well discussed in the official NASA podcast *Houston, We Have a Podcast*, episode #75, Radiation Shielding.

Uninformed user protection

There are many functionalities that can cause an undesired behavior. For example, simulating a key event without an active hook manager with a safe mode turned off can cause crash of the process. Specific pipeline setups can cause infinite loops. Setting up incorrect routing or using an insufficient password can compromise sensitive data or open the machine to attacks. Loading an external library gives the code full access to the system and the machine. Even when the issue of preventing errors is solved, there can still be potential risks for users who are not aware of them. Therefore, it is important to provide measures to protect such users from potential risks.

While one solution could be to simply limit functionalities of the system into few selected pre-defined use cases, this goes against the main goal of configurable and extensible system. Instead, it is proposed to use and expand the concept of expert-user and informed user decision. This system can be inspired by the Linux operating system, where users are members of groups with different permissions [22]. In this way, expert users can have unrestricted access to all functionalities, while more specific groups can be created for users with different levels of knowledge and access. For example, one group allows defining routing targets, another allows loading of user scripts, one for calling configuration commands, and another to run prepared command groups.

To support this, each command or user interface element should be assigned to one or more groups. Users are then assigned to specific groups by automatically called commands in configuration files or by user input, same as with hash values. Valid question now would be whether user names should be stored in plain text, hashed, or encrypted. Encrypting user names would require to also encrypt all the commands for assigning a given user into groups. Hashing user names would not require such changes but could cause potential issues with user management, such as transparency, simple direct communication, or recovery of forgotten user names. Storing user names in a plain text is the simplest solution, but it can be considered as a security risk.

The benefit of this approach is also that it reduces visual clutter for non-expert users, as they do not have to navigate through various options that they do not understand and that can cause potential risks. For expert users, this can provide the functionality of filter-based search of commands and options. Due to this, users can have more intuitive and direct control over the system, while being protected from potential risks [23].

Malicious attack mitigation

The remaining risks can be defined as a complement to the previous two categories. That is, any potential behavior that is not desired by the user and that is not caused by alteration of data or behavior of the system. Passive listening for communication between two machines can be considered as an example of such a risk. No detectable alteration of data or behavior has occurred, the system behaves as desired by transmitting data to the target, but sensitive data are being leaked to an unauthorized party.

To protect against such risks, it is important first to identify sensitive data. By careful consideration of the system, the following data can be defined as sensitive:

- Any data sent over network
- Configuration files
- Executed commands

- Generated input events
- Stored user information, specifically passwords

This list is not exhaustive, but it covers data that are specific to this system and would not be created without it. For example, user behavior patterns can also be considered as sensitive data, but can be captured at any point of user interaction with the machine, even when this system is not present. Generated input events are specific to this system, but as they are passed to other software, they will also not be further considered as sensitive data.

The design of a truly secure system would be beyond the scope of this thesis, but it is important to implement some measures to reduce potential risks. Following methods are not therefore intended to be unbreakable, but rather to not provide easy access to sensitive data for unauthorized parties. This follows the same approach as with other security categories, where the system provides some level of protection, but the final decision and responsibility are in the hands of the user.

To secure network communication, encrypting all data sent over a network is the most straightforward solution. Because the system does not need to interact with the data that is being sent, it can be easily encrypted and decrypted en masse without any complex management. The proposed idea is that the user provides a password for network communication. The message is first prefixed with a checksum of the message, then encrypted with the password and sent over a network. At the receiving end, the message is decrypted with the same password, a checksum is calculated and compared with the one sent with the message. If they match, the message is accepted; if not, it is rejected. This also gives the benefit of quiet rejection of messages sent by unauthorized parties, as they will simply fail the checksum verification and be ignored without any notification. The AES encryption algorithm and the MD5 checksum are believed to be sufficient for this purpose [18].

Although encryption of configuration files would be important, it is not currently implemented as it would deny manual editing of the configuration file, which is inconvenient for development and testing. As this would simply expand encryption of the hash value, this is not a technically interesting problem. The same applies to other files that might be loaded into the system. A simple command that allows encryption/decryption of any file protected by a user password or a dedicated file password would be sufficient.

The final aspect to consider is a protection of user information stored in memory, specifically passwords. Storing such sensitive data in a memory in a plain form is considered a security risk. Currently, passwords are the only user information that is stored in memory, following text deals only with them, but could be expanded onto other data if needed. For passwords, there is no reason to store their full form; instead they should be stored as hash values calculated with a strong algorithm, such as SHA-3. But storing hash values can still be considered as a security risk, as in C# language they often remain in memory longer than needed due to garbage collection. They are also moved around when the operating system is trying to optimize memory usage. To mitigate this, the hash value of a password is encapsulated in a class, together with a randomly generated mask value. The constructor of the class calculates a hash value of the password, generates a random mask value, and then applies a bitwise XOR operation between them. When such a short data sequence is XORed with a random value, it is practically indistinguishable from a random data [19]. In this way, the stored hash value should be protected from being identified in memory. Additionally, the class is designed to minimize data manipulation by unmasking the hash value directly into a target memory location.

Chapter 4

Implementation

The following sections are dedicated to the selected details of the implementation of the system. Familiarity with the general design as described in Chapter 3 is assumed. The emphasis in this chapter lies in the practical realization of those design aspects where platform constraints, performance considerations, or other implementation challenges required special attention. It should be noted that the C# language and .NET 8 were chosen for the implementation, primarily due to good Windows integration, fast development capabilities (also due to personal familiarity), and sufficient performance with option for critical parts to be implemented in lower-level languages such as C++ if needed [30]. The details discussed in this chapter are not specific to the chosen language or .NET platform, and can be adapted to other languages or frameworks with various levels of effort.

4.1 Windows Input API

To fully satisfy the needs of the InputResender system, 5 features should be provided by the chosen API. All of these features are desirable, but one of the first three is sufficient to provide some useful functionality. The last two are nice to have, but they are not strictly necessary for the system to function. These features are:

1. Read event values such as type of input action, key codes, or mouse movement deltas.
2. Determine the specific device that generated the input event. Allowing different devices of the same type to have different behaviors is a useful feature.
3. Generate custom input events. Interpreting received or transformed input events as if they originated from a real device improves the capabilities of the system.
4. Operate independently of the focused application. This allows the system to function globally, transforming an input for any application.
5. Consume input events before they reach the focused application. Without consumption, the original input event is still delivered to the original target, without any transformation. This leads to potential duplication of input actions.

These features are not specific to Windows only. Implementations for other operating systems should be able to provide similar capabilities, although the specific APIs and methods may differ. However, it should be noted that some platforms may have set up restrictions on what can be achieved with user input, as it can be used for malicious purposes.

For example, on mobile platforms, applications are typically sandboxed and have limited access to system-wide input events, which may limit the capabilities of the InputResender system on such platforms.

The Windows operating system provides multiple ways to work with input events. Security and stability play an important role in both the design of these APIs and their practical use. Apart of that, simplicity of implementation and performance are also important factors. The options provided are as follows:

- **Input Hooks** — As part of a Message Loop architecture, Windows provides a way to install *Hooks* that can listen to various system events, including input events. In their basic form, a hook is notified only about input events destined for the application that installed the hook. This event can be modified or even consumed, preventing it from being passed to the next handler in the chain. Such events can also be virtually generated and injected into the message queue of the application, although such events are marked as *injected* and some applications may choose to ignore them. Device specific information such as device ID is not available through this API. This approach satisfies requirements 1, 3, and 5.
- **Low Level Hooks** — Slightly more advanced version of input hooks are *Low Level Hooks*. These hooks operate at a system-wide level before the input event reaches the target application, allowing seamless transformation of a system-wide input, at the cost of specificity of the target application. With just a minimal increase in complexity, this will now satisfy requirements 1, 3, 4, and 5.
- **Raw Input** — For situations where Low Level Hooks are not low-level enough, Windows provides an option to read data from *Human Interface Devices (HID)* directly using *Raw Input* API, before it is processed by the operating system into message queues. This opens up the possibility of reading from specific devices, satisfying requirement 2. Raw Input is sent to a specific process that is registered to receive it, which means that it is not possible to consume or modify the input event before it reaches the target application. In fact, OS creates both Raw Input events and standard input messages from the same physical input event, fully independent of each other [17]. For these reasons, it is also not possible to generate custom raw input events and send them to target applications. This satisfies requirements 1, 2, and 4.
- **Device Drivers** — The lowest level of software control over input devices is through device drivers. When done properly, this approach can satisfy all of the requirements, as it receives data directly from the hardware, without any interference from the operating system. Furthermore, it is possible to create virtual devices that present themselves as real HID, allowing to generate custom input events that are indistinguishable from real ones. However, this approach is complex, time-consuming and prone to errors that may compromise not only the stability of InputResender or other applications, but the entire operating system or even introduce security vulnerabilities. Device Drivers are also very hardware dependent and are nearly impossible to be portable across different devices or operating systems.

After careful consideration of these options, it was decided that Low Level Hooks will be used as the primary method, sacrificing device specificity. Raw Input is considered a great future extension to provide extra features such as additional hotkey options or per-device selective behaviors. This decision is based on the fact that Low Level Hooks provide most

of the desired features with relatively low complexity and good performance, thus being a good compromise between capabilities and implementation effort [29].

This means that no ideal all-in-one solution is available, and some features will always be missing. The closest one to an ideal solution would be to have a custom drivers for all the devices involved, which is very complex, not portable, and potentially dangerous. Another option is to combine Low Level Hooks, Raw Input, and virtual input devices, which is as close to ideal as possible, but also very complex to implement. Under this setup, Raw Input captures input and stores it. Low Level Hooks are used to allow consumption of the input events that were captured by Raw Input. Finally, virtual input devices are used to generate custom input events that are almost indistinguishable from a real ones. Note that this setup seems to satisfy all of the requirements, but this is not guaranteed. First, any application listening for Raw Input events would still receive the original input event, even if it is consumed by Low Level Hooks. Second, some applications may choose to ignore virtual input events, as they might still lack some specific information that real input events have, such as an ID of the manufacturer or the device. Finally, Raw Input events and Low Level Hooks events, even when created from the same physical input event, are still independent of each other, which means that they might not be perfectly matched, leading to potential issues with synchronization of the system.

4.2 Scripting

As discussed in Section 3.4, scripting should play an important role in the interaction with the system. The need for sandboxing and high performance limits options for using existing solutions with simple embedding. As scripts should be able to perform complex logic in real-time, if nothing else, to determine whether to pass the input event or consume it, performance is critical. While for C# there are packages providing interpreters for popular languages like Python, these solutions can be outright rejected due to performance concerns. Interpreting scripts in real-time is considered to be unnecessary waste of resources.

More tempting are solutions that compile the scripts into a more efficient representation before execution. For C#, the Roslyn compiler provides great capabilities to compile and execute C# code dynamically at runtime. This approach offers excellent performance and familiar syntax for C# developers. The main issue is problematic sandboxing, as whatever code is compiled and executed, it has all the capabilities of the host application. This could be mitigated by performing static code analysis during script loading, denying any code that uses tokens outside of a predefined safe list. During runtime, the executed script would be given only those functionalities from the InputResender that were explicitly requested, as proposed in Section 3.4. Along with a high possibility of mistakes during static analysis, a high level of abstraction would be needed to provide safe and correct access to InputResender functionalities, making this approach complex and error-prone.

The third option is to use a custom interpreter for either an existing or custom scripting language. A major benefit of a custom interpreter is a full control over the executed code, making sandboxing trivial. With careful design, sufficient performance can be achieved, with plenty of room for future optimizations as needed. Depending on the complexity of the chosen language, the implementation effort can be relatively low. Designing custom language also opens up possibility for specifically tailored syntax and script complexity reduction. The downside is the complexity of implementing the interpreter itself, the need for users to learn a new language, and potential performance issues with larger scripts.

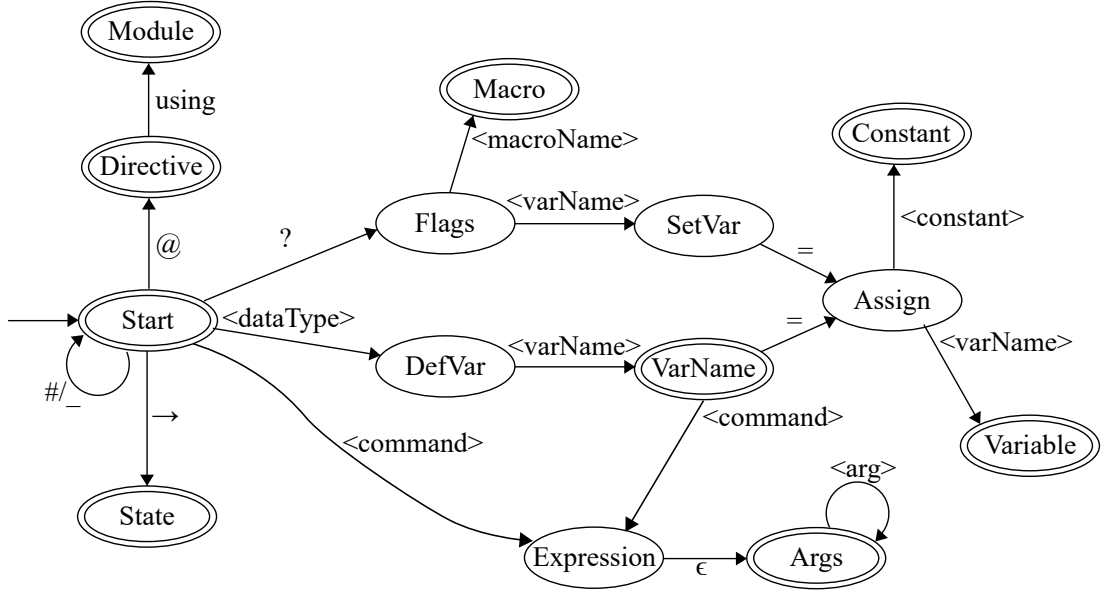


Figure 4.1: Proposed finite state machine for parsing individual lines of scripts.

As the feature is at the time of writing under active development, the required extent of scripting capabilities is not fully known yet. As the complexity of embedding and securing an existing solution was deemed to be as high as implementing a simple custom interpreter, it was decided in favor of the custom interpreter. So far, this approach has shown good potential and great flexibility.

To keep the implementation effort low, the language should remain simple. Syntax is kept to the bare minimum, allowing for finite state machine (FSM) based parsing without the need for a complex grammar analysis. The proposed FSM is shown in Figure 4.1. To allow for personalized expansion of the language in individual modules, any semantics is provided by the modules themselves, with the core interpreter only providing basic constructs such as variable assignment or conditional statements. Contributing modules are specified at the beginning of the script, dynamically loading operators, functions, data types, and other constructs during compilation.

Using a pure finite state machine for parsing lead to slight issues with debugging and also resulted in higher code duplication. Therefore, a simpler and more direct approach of sub-parsers was used, where line is attempted to be parsed by multiple sub-parsers, each responsible for specific construct, until one of them succeeds. Parsing is first done into an intermediate representation, which is applied to the current parsed script state only once the entire line is successfully parsed. This allows for easy re-use of code, better error checking, and more straightforward debugging.

The script is parsed into a linear list of instructions, which are then executed sequentially. Variable names are converted into an index in an array of pointers to variable structures during compilation, allowing for a fast variable access during execution. Each instruction call is implemented using a structure aligned to 16 bytes for efficient CPU caching [25], which is designed as shown in Table 4.1.

To define the control flow, two techniques are used. First, each line can be prefixed with up to 3 flag marks, which must all be raised for all the instructions generated from this line to be executed. This allows for a simple implementation of conditional statements. Flags

Offset (bytes)	0-1	2-3	4-5	6-7
Meaning	OpCode	Destination	Operand 1	Operand 2
Offset (bytes)	8-9	10-11	12-13	14-15
Meaning	Operand 3	Operand 4	Flag Mask	Extra Data

Table 4.1: Structure of a single instruction in the script interpreter.

are stored in the runtime environment and can be set by any instruction. Up to 16 different flags can be used, first 6 are reserved for comparison results.

Second, to avoid infamous *goto* statements, an alternative approach to unrestricted jumps was considered. Because the script is mainly designed to implement input event processing logic, such behavior is strikingly similar to state machines. But enforcing strict finite state machine structure might lead to unnecessary complexity, especially for simple scripts. Therefore, a compromise solution was designed, where a state machine structure is only a strict syntactic sugar for jumps. The user can define at any point in the script a state with a name and all the transitions that this state can initiate and which virtual tokens initiate them. Anywhere after the state definition until either the next state definition or an end of the script, a user can call *emit* macro with one of the virtual tokens.

During compilation of the script, the definitions are stored in a temporary compilation state. Emit calls are replaced with either a *Jump* instruction for sequential targets or a *Split* instruction for parallel targets. During execution, the Jump instruction simply sets the instruction pointer to the target state, while the Split instruction queues the target state for execution in a set of active states and continues with the next instruction. Although parallel execution is not implemented at the time of writing, the planned mode of execution is that, depending on an implementation of the executor component and its settings, the queued states are either executed once all the active states are finished or suspended, or they are executed immediately in parallel within separate thread.

If some state was already defined and another state definition is encountered, one of the other two instructions is inserted at the end of the previous state. For accepting states, a *Terminate* instruction is used, which stops current execution and enqueues the entry (i.e. the first) state into a set of inactive states. For non-accepting states¹, *Suspend* instruction is inserted, which enqueues start of current state into set of inactive states. This allows for simple restart of either an entire script or the most recent state. Motivation is an example scenario of receiving a sequence of input events. If the sequence contains some unexpected events, those can easily be skipped by restarting the current state, and finishing after all expected events are received, with an option to start again for another sequence.

When a new runtime environment is created, active program counter is initialized with the first instruction of the script, ignoring anything related to the state machine. Thus, simple scripts do not have to be unnecessarily complicated by the state machine structure, and also providing a logical space for data initialization and other setup. Local variables are not currently supported, but when implemented, they will be created or deleted along with resetting flags once either a jump is performed or a new state is picked from the queue of active states. When an active state runs to the end and no more active states are queued, the execution stops and control is returned to the caller. The script can be executed again,

¹The terms *Terminating* and *Suspending* state would be more fitting here, but we will keep the *Accepting* and *Non-accepting* standard terminology here, as the system is only heavily inspired by finite state machines, not a complete adaptation.

which will transfer all inactive states into active ones, or inserts the first instruction of the entry state into active states. Note that this design ensures that instructions before the first state definition are executed only once, unless the environment is recreated anew.

At the moment of writing, scripting capabilities are still very limited due to very few implemented modules with instructions and data types. In terms of the language design, the definition of data containers and routines is planned, although the specific syntax and capabilities of these features are not yet determined. However, even with current limited capabilities, the system is already able to implement some useful scripts, such as Morse code emulation or conditional routing. Performance of these scripts is good, with no noticeable input lag, see Section 5.1 for more details.

For more complex scenarios, a mix of custom scripting for an overall behavior of the script, with a Roslyn compiled C# micro-snippets is being considered. Delegating mathematical computations into existing interpreter could provide not only a performance boost, but also access to familiar rich syntax of existing language. Supporting this should keep static analysis simple enough to avoid security issues. This feature is still in an early consideration phase and is not implemented at the time of writing.

4.3 Console Commands

As proposed in Section 3.4, the system is mainly operated by console commands. The processing is handled by a dedicated component registered in a core, at the same level and with the same restrictions as any other component, namely that it should be oblivious about other parts of the system apart of a direct connection to its core owner. All console commands must be registered from the outside of a command processor. The most essential commands, such as *help* or *loadall*, are automatically registered by the virtual CLI. Other commands are organized in a forest structure under command loaders. Command loaders are based on the factory pattern, allowing for dynamic registration of new commands, replacing existing ones, or expanding them with sub-commands.

Each command is implemented as a separate component, which allows great flexibility in their usage, mainly the pipelining system can thus be restricted to components only, while also allowing commands to be a part of or the initiators of pipelines. Commands can also now be accessed via core fetches, enabling them to interact with each other more directly than through the command processor. Additionally, commands that exist only to control the state of a specific component can be merged directly into this component, without the need to create a separate object.

Each command follows the same structure. Deriving from a component definition, it must be registered into the core, has a common name, is versioned, and can produce snapshots of its state, same as any other component. In addition to that, they must provide a call name, which is used to invoke the command, and a list of sub-commands or general parameters. When a command instance is selected by the command processor to be executed, it must first respect a potential request for a help message. After that, it can load parameter values from a pre-parsed user input, among with any additional setup, such as fetching controlled components or other commands. Based on sub-command selector, if one is expected, execution will follow one of the switch branches. Finally, command returns textual status message, such as a confirmation or warning, with an option to return some data as well. Result data can be used to start pipeline execution or to pass it to the caller when used as an API call. Argument parsing is provided by the global service which is also used in other parts of the system, such as parsing of macros in user scripts,

allowing for consistent syntax and behavior across the system. This service also provides some basic data type parsing, such as numbers, strings, or enums, with planned support for more complex data types in the future, probably based on the *JSON* format.

To further reduce coupling with other parts of the system, command processor offers a dictionary based storage for public data, which can be accessed behind the scenes during command execution or by explicit user manipulation. This means that while command processor is registered under some core, it is not restricted to interact only with this one. Any other existing core within the system can be assigned as the active core to change the context of command execution. While this specific feature is not currently used, it opens up interesting possibilities for future development. Complex setups might benefit from running multiple cores under the same process, or, if properly implemented, it could allow controlling cores on other machines over the network.

All commands are designed to be executed in a query-response manner, which means that they are executed once per a user input and then return control back to the user, together with any output. Commands are not restricted to producing some long-term effects, such as starting a new thread, registering new components, or installing callbacks. The modus operandi in such a scenario is to call one command to start some long-term effect, and then call another command to either request status of this effect or to stop it. When some output is produced outside of the query-response manner, such as in a callback, it is buffered as a *delayed message*, which can be retrieved on demand by user command, or can be periodically written to the console either as part of message loop of operating system or before any new command prompt. The only exception is, if the user explicitly desires a specific callback to write directly to the standard output, in which case it uses the virtual CLI that might or might not be directly connected to the real console.

To further expand the options of this system and allow adaptation to various use cases, the virtual console is equipped with smart output formatting and input masking. The output can be displayed as is, can be merged into a single line with the entered command if it is shorter than configurable threshold, or can be bundled into a batch format, with specific separators and metadata to be well understood by external parsers.² Input masking allows for hiding any part of the user input, such as passwords or other sensitive data. Any text between specific delimiters, either from user input, generated output, or received console data from other sources, can be masked out. This allows the user to specify themselves what data are sensitive on a per-command basis.

4.4 Provided Input Transformations

To provide some options of user input transformation out-of-the-box, there are 4 implemented options. One of them is a direct passthrough. Even with this simple functionality, some processing needs to be performed. Because the *InputProcessor* component expects an array of user input structures which represents the most recent user input alongside a relevant context, it should convert it into an *InputData* structure, expected by other components as their pipelined input. That includes converting the context into a proper input command, such as a key press or a mouse move, and extracting a set of all active modifier keys. Those typically include keys like Shift or Control, but can be configured to support virtual user-defined modifiers, for example, key E to set virtual modifier #3.

²The batch format is currently not used, but it was used during experimenting with Java-based client and .NET-based server for Android devices.

The *ScriptedInputProcessor* variant can execute a user script on received input events. Because user scripts are parsed and owned by a dedicated command and because components should not rely on holding any references to other components, parsed script must be assigned dynamically by fetching active instances of both script manager command and the scripted input processor. If system status changes in such a way that the previous reference breaks, user can restore the state by calling the assigning command anew. Whenever a user assigns new script instance into the input processor, it will create a new runtime environment. Unlike setting a new one on every single new input event, this not only saves some setup time, but most importantly allows to restart the script from last state, if it finished on a non-accepting one.

The first step in processing is filtering. The processing component can be assigned some accepting keys or input actions, either by a command by the user, or by a user script executed previously. The scripted input processor will therefore first analyze the input event sequence and checks if the input fits the provided criteria. If not, rejection of the input is immediately returned, marking that the input event should be passed on to another link in the hook chain of the operating system. Otherwise, the processor attempts to set supported potential external variables of the script, such as a status reference or whether some setting of the component has changed since the last execution. These external variables do not have to be contained in the user script.

Script execution is now requested, which will pause the transformation processing until the script interpreter no longer has any states in its active set. After the script execution halts, the component will read any of the supported output values, which, at the moment of writing, there is only a single boolean value, indicating whether the processed event should be consumed or passed on. To provide access to the functionalities of the processor for the script, a dedicated module is provided. This module provides instructions to extract information from the status structure, to request firing a new input event, and to change input filtering, among others.

As an example of a user script that can be started by this component, a Morse code emulation was implemented. This script will automatically configure filters so that only three keys are captured and processed for the dot, dash, and a character separator. Any other user input is ignored and passed on to the operating system. When either the dot or dash key is pressed, the script will advance to the next state, suspending at the end of that state. Once the character separator key is pressed, the key associated with the current state is fired and the script transitions to an entry state.

An example of more complex processing done by a dedicated component is the *Tap-perInput* component. This component maps combinations of up to 5 pressed keys onto a specific output key, inspired by the wearable keyboard product *Tap Strap 2* [21]. This input method can provide a sufficiently fast typing speed with only one hand, without the need for any hand movement. In the original example of typing while playing a video game, the user can use the ASDF and Space keys for typing when a side mouse button is pressed, allowing very fast switching between typing and moving, without affecting the aim or any mouse actions at all. To support this, the component can be assigned a trigger modifier, which can be either a real system modifier or a virtual one. If this trigger is not active, the component will ignore all input events and pass them on to the operating system. Once the trigger is active, the component will listen for both key down and key up events. On a key down, the component will simply recalculate the index into a mapping table and consume the original event. On the first key up, the component will fire the key associated with the current index, reset the index to 0, and wait for key up events for all

still pressed keys. Unlike in the original product, where due to hardware reasons, all fingers of the intended combination must tap some hard surface withing a short time window, this software implementation does not put any time constraints on the input, which can be beneficial for users that are still learning the combinations.

Throughout development, other components for input transformation were also experimented with. That includes *TextWriter*, which would store written text, allow simple modifications such as undo and redo operations, and could be entered as a finished string on demand. *EventVector* was designed to support assigning and raising callbacks based on string names, or an *ShortcutWorker* which would be raising callbacks based on entered input combinations. All of those components were finally deemed to not be useful enough and that similar behavior can be achieved by user scripts, and their development was discontinued. The idea of a component for general post-processing of the *InputData* structure came for unexplainable reasons way too late in development to be implemented and tested. Such component, which would implement a specific transformation or start a user script, and could be chained multiple times in the pipeline, might be a great addition to the system.

4.5 User Interface

In the traditional Model-View-Controller (MVC) architecture [11], the command processor with a virtual CLI can be seen as the *Controller* part. It serves as the main actor that controls the behavior of the system. Although this could provide the user with some interface to interact with the system, it is lacking in efficiency. As the system grows in complexity, so does the number of commands and their parameters. This approach also works mainly in a query-response manner, reducing the options for real-time interaction. For these reasons, some dedicated user interface should be provided.

For the *View* part, many options are available. As it is desired to have a cross-platform solution, a web-based *Graphical User Interface* (GUI) is considered a promising option. And because the system is being developed in a C# language, Blazor framework was chosen [3]. It should be noted that not only could other UIs be implemented along with the Blazor system, but the system is designed to allow multiple UIs of the same or different types to be active at the same time. Only limitations in this regard would be potential performance issues and possibility of data-race. These limitations are not caused by multiple UIs, they could be present even with the basic variant of a single CLI, their probability and severity increases with the number of active modules, of which the UIs are the most demanding ones.

The Blazor GUI is implemented as a separate module. Initially, without any running instance, it must be created by a dedicated command. Using the capabilities of the Blazor framework, it is started in a separate thread, leaving the option of CLI interaction available in the background. The performance aspects is not discussed in this text, as a single user connected to localhost is expected. In such a setup, the difference compared to other GUI solutions should be negligible. It should be possible to convert the entire system into a Client-Server architecture, where routing or transformation or both are provided by a centralized hub. This would require a very different GUI and large changes to the system as a whole. Such a large-scale product would require a deeper dive into security, optimization, and networking, and would be best left to a large professional team, although the basis that is laid down in this project are deemed sufficient even for this type of a product.

A challenge in the design is presented by the requirement for a platform-agnostic modular architecture. As the ignorance should work in both ways, which means that the UI

should not be aware of other components and components should not know how the UI works; presented pages must be generated dynamically based on the provided generic UI objects. This is solved by adding another layer of abstraction. Therefore, to serve as the *Model* part of the architecture, all components, commands, and scripts are capable of providing objects with relevant data. This should be as platform-agnostic as possible; thus, only a generic type of *UI object* is provided. These are mostly wrappers around basic data types, such as strings, numbers, lists, or actions, with additional metadata such as name, label, or description. Any UI system can then use these objects to generate an appropriate graphical visualization or better textual system.

For each generic UI object, a specific Blazor component is created, which is responsible for rendering the object and providing all the necessary interactivity. Although this provides great flexibility with dynamic UI generation, it also serves as a limiting factor for the design of UI objects, as a dedicated page for specific functionality cannot be hand-crafted. This could be improved by expanding the UI object system to provide more information how the individual elements are related, allowing for more complex page generation. Because the current system provides sufficient capabilities, this improvement is left for future development, as other requirements are deemed more important at the moment.

Using this system, the UI is decoupled from the console system. Instead, when UI descriptions are used as a controller, they can be updated directly by any component immediately when the state changes, allowing for a real-time display visualization in GUI. Access to these data can also be requested from other elements specific to one given platform, such as a Blazor component, which can manipulate other UI elements, rather than via console commands. Some UI objects are by nature difficult to port across different platforms, such as 3D visualization or interactive graphs. In such scenarios, a dedicated GUI element can be created, hopefully with much higher user-friendliness than a simple combination of provided UI primitives.

Expanding on this idea, when using the Blazor GUI as an example, there should be a set of dedicated Blazor pages for a specific common use cases. For example, a single page application to set up a common input processing pipeline and a user-friendly dashboard of important data. Although this is not specific only to Blazor, rather to any 2D GUI system, but at this moment there is no need for more complex UI, that could not be implemented just as an additional component with some UI definitions. But it is expected that focus in development shifts towards more user-friendly features. The same goes for other platforms, where, for example, console-based UI could implement a dedicated interactive menu system, or a 3D game natively supporting this system could make use of this approach to provide a dedicated in-game UI or even base some of its gameplay mechanics on the system.

Chapter 5

Evaluation

As there is no one specific use case, rather there are many use cases with multiple valid workflows, a formal verification of the system would be challenging. Instead, a suite of automated tests were created to verify that the system behaves as expected under *normal* conditions. As is often the case, this cannot guarantee that the system will always work correctly, but can provide some level of confidence in the implementation.

In addition to extensive unit tests for individual building blocks of the system, each component definition provides a set of test cases covering both typical usage and potential edge cases. Each implementation of the component, including the most basic mock-up implementation that is provided together with the component definition, is expected to run and pass all of these tests. This demonstrates that the generic behavior is consistent across different implementations. Additionally, integration tests are checking that the components can work together as expected. Finally, a set of system tests are provided to ensure that the system as a whole operates as expected from the user's perspective.

Manual testing was also performed to ensure the correct behavior of the system in real-world scenarios. This was also the main method for testing the correct behavior when transmitting user input to different target machines. Specifically, following scenarios were tested:

- Converting E/T/R keys as a dot, dash, and character separator in Morse code input, re-simulating the input on the same machine.
- Selective redirection. Numpad Enter switches between local and remote machine.
- Encoding F20-F24 key combinations as regular input on the remote machine, while playing a video game on the local machine.¹
- Changing contents of configuration or script files, which was correctly detected and confirmation to continue was requested.
- Load a DLL with an additional component and command, and use those to manipulate clipboard content.

¹This was the main use case of the project. The game used for testing was *Fall Guys*, as it requires fast real-time input, has simple enough controls to be controlled with mouse only, and potential ban for using software that can automate input is not a big concern.

5.1 Performance Testing

Because good performance is one of the goals, as mentioned in Section 3.1, performance testing is deemed to be an important part of the evaluation. Testing was done on two representative machines:

- Gaming PC: AMD Ryzen 7 7700X3.5 GHz, 64 GB RAM5600 MT/s, Windows 11 Pro, Azeron Cyro Mouse, Razor Tartarus Pro Keyboard
- 2-in-1 personal laptop: AMD Ryzen 5 8640HS3.5 GHz, 16 GB RAM6400 MT/s, Windows 11 Pro, built-in keyboard

Both machines were used in high and low utilization scenarios. To achieve high utilization on the gaming PC, VirtualBox with Linux Mint guest was used, together with HandBrake video encoding and video games running on the host machine. This results in >90% utilization of CPU and RAM. On the laptop, JetBrains Rider IDE and 7-Zip file archiver were sufficient to also achieve >90% utilization of CPU and RAM. These processes were stopped for low-utilization testing. This results in <10% CPU utilization and <40% RAM utilization on both machines.

5.1.1 Latency

Latency is the most important performance metric for the system. Any noticeable delay caused by using the system would render it unusable for many use cases. Therefore, it is important to ensure that the system does not introduce any noticeable delay. It should also be tested on a machine under high load, to ensure that the system can perform well even when the machine is under stress, such as while gaming, video editing, or with virtual machines and other resource intensive applications running in the background.

First, a simple subjective test was performed as a part of manual real-world testing. The system ran in the background for one week, connected to a remote machine over LAN. Local machine used the *TapperInput* component to remap F20-F24 keys on separate gaming keypad to standard keyboard events. Other input events from the gaming keypad were passed back to the operating system without any modification, same as main keyboard and mouse. On the remote machine, received transformed input was re-simulated as regular input, which was used to write messages and other text in various applications. During this test, the local machine was used for regular work, including gaming, same as was the remote machine. The remote machine was also disconnected and reconnected multiple times, hibernated, and put to sleep. Not only there was no noticeable delay when the planned testing was performed, but in fact it was a surprise multiple times during the following days that the system was actually still running and connected. This was a good indication that the system is performing smoothly and without any significant latency.

Following the subjective test, a more thorough and objective test was attempted. To test the most inner and most critical part of the system, timestamps were added into the methods for simulating user input and hook callback. This would allow to measure the time delay caused by the additional hook, if the event is not processed by the system in any way.² This initial setup did not provided any measurable delay.

²This is the scenario when the lowest possible delay is required. For example, user enables transformation and/or routing between game matches, but wants to have anything to be disabled during the matches to not lose competitive edge.

To expand this test to provide usable results, first, the Stopwatch class was used instead of a DateTime, which should provide better precision. Second, time measurements were added to methods for input processing and routing, to gather more data on various scenarios. Third, the scenario was expanded so that it is fully processed with simple remapping of keys. There was hope that this would result in a sufficiently significant delay, but it was still not noticeable compared to network latency. Therefore, it was decided to extrapolate the results from a throughput testing, which will be described in the following section, to obtain an estimate of the latency for different scenarios.

5.1.2 Throughput

Throughput is another important performance metric, as it can indicate how well the system can handle a high load of events. In real-world use cases, not more than a few events per millisecond are expected. Even if modern gaming mouse can work above 1000 Hz and when working with multiple such devices, that leaves an upper bound of approximately thousand events per millisecond.

The main idea for these tests is to skip the real Windows implementation of input hooks, use mockup components instead, and flood the system with a large number of events. If *fast* hooks are used, this will result in sequential processing on single thread and thus will provide as accurate results as possible with minimum outside influence. *Slow* hooks would introduce problems with thread synchronization and would result in higher testing complexity. Using Windows hooks would give more realistic results, but would introduce significant amount of noise and would also increase complexity of testing, as events would no longer be guaranteed to be captured sequentially.

Because such a design executes all processing on a single thread, individual events are processed sequentially. Also, at the moment, no buffering, batching, or other optimizations for streams of events are implemented. Therefore, a relation between the number of events and the time taken to process them is expected to be linear. This allows extrapolating the results to obtain an estimate of the latency as an average time taken to process a single event. Although this does not provide any variance or worst-case estimates, it still provides the average latency of the system, removing outside interference and noise.

There are 6 different scenarios that were tested, each representing different use cases that do partially overlap with other scenarios. Complete isolation is not possible without modification of the system itself, but the results provide good insight into the performance of the system under different conditions. These scenarios are as follows:

1. **Simulate-Hook** — Internally simulate predefined key press and capture it via hook. This serves as a baseline of the overhead caused by the test setup itself, as these are operations carried out by the operating system in real-world use cases.
2. **Mock Pipeline** — Execute the standard processing pipeline with a mockup input processor, implementing only the basic passthrough behavior, without any routing. This will provide insight into the overhead caused by the pipeline system, which serves as the main communication path between components.
3. **Commands** — Execute the Simulate-Hook scenario, but with console commands enabled. This will provide insight into the overhead caused by the string parsing and command execution. This is a scenario where the console is used as an API for external applications, as described in Section 4.3.

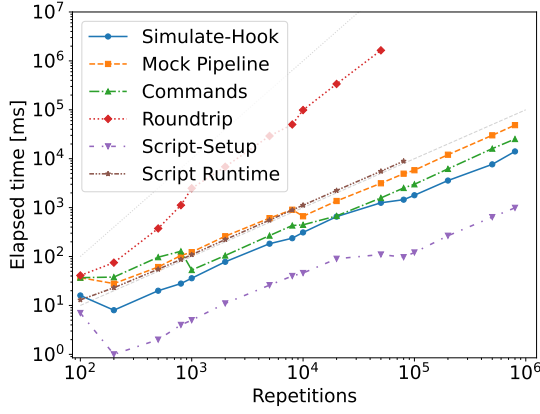
4. **Roundtrip** — Run the full pipeline for the sender and receiver, mocking the network communication between them. This estimates the overhead of the full basic system.
5. **Script Setup** — Construct runtime for user script, set input variables, execute basic script, and read output variables. This will provide insight into the overhead of creating new script runtimes and accessing the data from them. These operations should be as rare as possible, ideally only during initialization, but it is useful to know if they can be used more often if needed.
6. **Script Runtime** — Re-evaluate complex user script, with multiple instructions and states. This test will reuse the runtime created during an initialization phase to separately evaluate the overhead of the execution itself. The script consists of 880 instructions, split into 5 states, each state executes 10 times. Script has no input or output variables. Currently, there are no optimizations of the compiled script, so this scenario can estimate the average time taken to execute a single instruction.

These tests were performed with an increasing number of events, starting at 100 events, and increasing up to $8e5$ events or until a timeout of 15 minutes is reached. This allows to see how the system performs under different loads and if there are any bottlenecks that need to be addressed. These numbers of events were chosen to eliminate noise as much as possible, while still giving good scalability estimates.

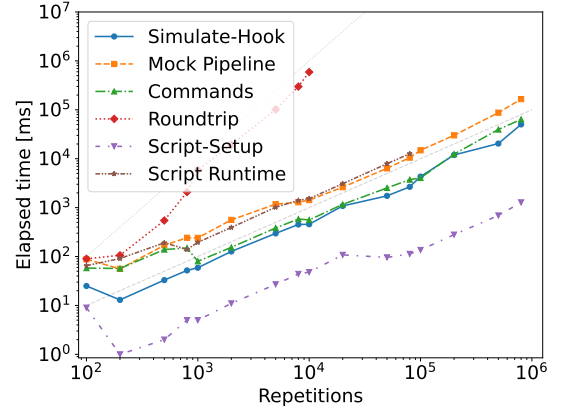
The results of these tests are shown in Figure 5.1. Most of the scenarios do indeed follow the expected linear growth. An outlier is the **Roundtrip** scenario, which seems to grow quadratically. This is probably caused by memory leak, as the memory usage was observed to grow from 40 MB to 15 GB during each of the scenario executions. This leak appears when some of the input processing components are involved, most likely some hardcoded logging for debugging purposes, which is not properly cleared after processing each event. Although this can be considered to be a serious issue, memory usage during real-world manual testing did not grow above 100 MB over more than a week of various usage of both the system and the machine. This indicates that this leak does not affect real-world usage until very high input rate. For such situations, other optimizations, such as buffering and batching of events, would be useful to implement along with fixing the leak, to further improve the performance of the system under high load.

Although there is some difference in performance between different machines and utilization levels, the differences are not significant, and the overall performance of the system is good even under high load. This could imply low hardware utilization, possibly due to use of Object oriented design and related overhead of virtual method calls. Better performance would be a nice to have, but the development comfort and maintainability of the codebase is more important, especially considering that the system is already performing well enough.

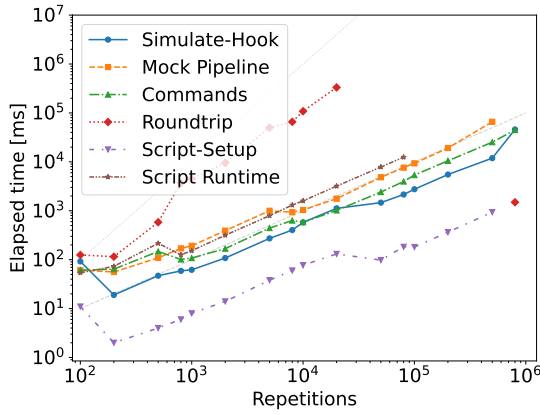
The good results are with respect to the execution of the user script and the setup of runtime. The runtime setup takes only minimal time, which allows one to use it as often as needed, without worrying about performance. Script execution is also reasonably fast, with average time of $0.5\mu s$ per one instruction. Even though user scripts are not expected to do any heavy processing, rather simply call specific instructions that are implemented in C# and let them do the heavy lifting, this is still a good result, as it allows to use user scripts for more complex logic if needed, without worrying about performance. It is worth noting that both of these scenarios, i.e. script setup and execution, do not overlap with the other scenarios, and thus can be added on top of the other scenarios to estimate how the overall performance would be affected when user scripts are used.



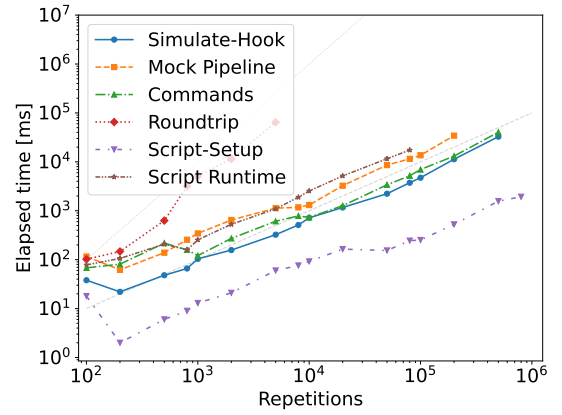
(a) PC - Low utilization



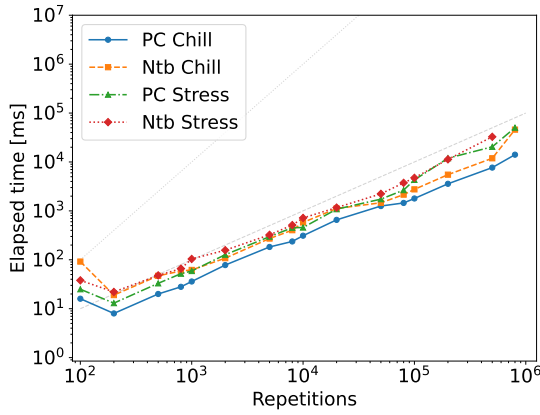
(b) PC - High utilization



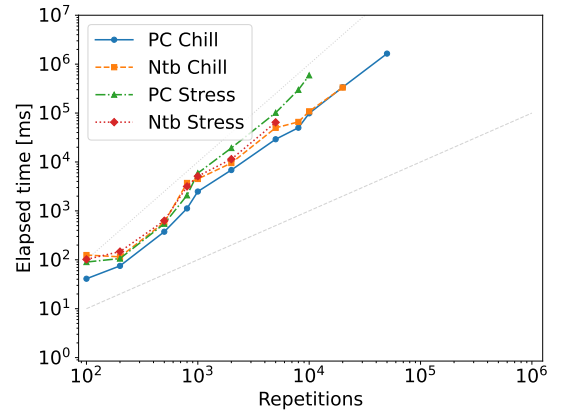
(c) Laptop - Low utilization



(d) Laptop - High utilization



(e) Simulate-Capture



(f) Emulated roundtrip

Figure 5.1: Throughput testing results on different machines and selected scenarios. (a)-(d) show results for all scenarios on different machines and utilization levels, while (e) and (f) show results for Simulate-Hook and Roundtrip scenarios, which are the most important ones, for all machines and utilization levels. There are two reference lines for linear and quadratic growth to help with distortion of the plot due to logarithmic scale.

To estimate the average processing delay, all machines and utilization levels were considered together. The minimal simple Simulate-Hook scenario, which serves as a baseline, takes on average 0.022 ms per one event. For the more complex Roundtrip scenario, which serves as an example of a real-world use case, one event took on average 0.87 ms to process. As for the basic scenario, this presents a very good result, where even if the event is consumed or passed over at the end of the basic processing pipeline, the delay is still very low to be noticed even when a low delay is crucial. While the Roundtrip scenario takes significantly more time, this also includes encryption and decryption of the data, which presents significant overhead that is not present during local usage of the system. Although there may be real-world scenarios for full roundtrip, the typical use case is to do local processing and do the encryption and transmission as part of the *delayed* callback, which is done in a separate thread, and thus it does not contribute to the perceived latency of the system.

5.1.3 LLM Compatibility

With an ever increasing popularity of large language models as tools for software development, it is important to evaluate how well such assistants can operate with the codebase of any given project. Because one of the goals of this project is for the system to be easily expandable, it is important to evaluate how well LLMs can understand the codebase and generate new components or modify existing ones. This also serves as a good metric for the general readability and maintainability of the codebase. If an LLM can work directly with the codebase without the need for an extensive interaction with a knowledgeable developer, such code should also be easier to understand and modify existing code not only by software tools but also by human developers. Additionally, good compatibility with LLMs offers potential fast development of new features.

Although many of the building blocks in this project are complex and critical for correct behavior of the system, many of the higher level elements follow very specific patterns and are sufficiently simple to be easily generated by LLMs. This includes, for example, specific component variants, commands, or script modules. Component definitions can cause some problems, as their behavior is not yet defined and LLM can rely only on the description of the component by developers, which can be insufficient.

All other elements mentioned above have the advantage of relying on already defined components. For example, when creating a new component variant, LLM can be instructed to use existing component definition, or a provided mockup variant, and to simply change the internal process. As long as this process is simple enough, LLM should be able to generate the correct code for the new component variant. There is also a plentiful amount of code already written for other elements that can serve as a set of great examples, which should further help LLM to generate correct code.

Although LLMs were increasingly used during later stages of development, the generated code was closely reviewed and often modified by the developer. It was also used only for single elements, to not make the review process too difficult. Therefore, to test the LLM compatibility, it was evaluated on a specific task that will not be included in the final implementation, as it is not a planned feature.

As a test task, traditional phone keypad input was to be implemented. A numeric keyboard is transformed into an alphabetic input, with 2-0 keys representing different sets of characters. The traditional phone layout is used, rather than a layout based on key values, so the Num8 key is where 2 would be, so it represents ABC characters. Character is simulated when another key is pressed or a timeout occurs. This is a sufficiently straightforward

transformation, but requires a code complex enough and should be a good test of the LLM capability to generate an extension for the system. This requires the following steps:

- Create a new component definition for the phone keypad input, with appropriate configuration options, such as a timeout duration.³
- Create a new component variant for the phone keypad input, which will implement the actual logic for processing the input and simulating the output.
- Create a new console command to configure the phone keypad input component, such as enabling or changing the timeout duration and add it to a proper command loader.
- Create a new user script implementing the logic for processing. As the `ScriptedInputProcessor` component does not currently support timeouts, a new event will be simulated either when another key is pressed, when another Num key is pressed, or when a given time duration has passed since a previous key press of the same key. This will require a new instruction for time measurement to be implemented as well.
- Add new system test for the component version.
- Add new system test for the scripted version, using inline script.

For this task, the *Claude Sonnet 4.6* model was used, in an *Agent* mode, under the *GitHub Copilot* extension in *JetBrains Rider 2026.1.0.1* IDE. To implement this hypothetical feature, a prompt was given to the model in a single message, without any follow-up interaction, to test how well the model can work on its own, just with the initial prompt and without any help from the developer. The prompt was as follows:

```
Implement new functionality to simulate traditional phone keypad input.
Numeric keyboard are transformed into alphabetic input, keeping
the~visual layout, so Num8 key maps ABC, Num9 maps DEF, Num4 maps GHI,
and so on. Character is simulated when another key is pressed or
a~timeout occurs. Otherwise cycle through the~characters when the~same
key is pressed multiple times. Use #file:DInputProcessor.cs for
component definition example, and #file:VTapperInput.cs and
#file:VScriptedInputProcessor.cs for component variant examples. To
understand basics of the~project, I recommend #file:ProjectOverview.md
file and #file:Development.md for specific details about expanding
this project. There are expected to be following steps:
1) Create component definition for the~phone keypad input, with
configuration options such as timeout duration.
2) Create component variant for the~phone keypad input, which will
implement the~actual logic for processing the~input and fire events
when character is simulated.
```

³The step 1 is not actually needed, as the `DInputProcessor` definition already exists and can and should be used as a base for the new component variant. However, the documentation for this project mentions this as a first step, expecting a brand new component is being created. While this detail should be added to the documentation, the existence of multiple component variants deriving from `DInputProcessor` should be a sufficient hint for the LLM to use the existing definition and to skip this step. Therefore, this step will be included in the prompt as a test, as a developer not familiar with the codebase might also not realize redundancy of this step.

- 3) Create console command to configure this component, such as enabling or changing timeout duration and add it to proper command loader. Use `#file:VTapperInputCommand.cs` or `#file:ScriptedInputProcessorCommand.cs` (or any other command) for example.
- 4) Add system test with inline SCL script (as another test under `#file:InputTransformationTest.cs`) implementing the logic for processing. Instead of timeout mechanism, fire simulated event when different Num key is pressed or when same key is pressed after given time duration. This will require new instruction for time measurement to be implemented as well, so expand the `#file:SCL_Module.cs` with the new instruction. Make sure to use generic version of `SetExternVar` method to not cause issues with data type definition duplication. Check the `#file:SeClav.md` file for SCL scripting documentation.
- 5) Add another system test for the component version, also as another test case into `#file:InputTransformationTest.cs`

With a context window of 105K out of 128K possible tokens, the model was able to generate almost functional code for all of the steps. That includes 3 new files and 7 modified files, with a total of 506 new lines of code, 1 modified line. At the first glance, the generated code appeared to be a good starting point, with clear structure and patterns. However, the generated tests were not passing. Even though a fully functional code would be preferred, failing tests are still useful to warn about issues in the generated code. The tests closely followed other existing tests for similar components and did not seem to have any issues on their own. Upon closer inspection, the generated code had the following issues:

- The script expects that the first state is entered when the first key is pressed, but the execution does not stop before any state header is reached, which causes the first encountered state to be executed during the first iteration of the script. This can be fixed by defining the code before the first state as the first *Start* state, which will transition to some next state only when the first key is pressed.
- Missing 4th argument for the `COMPARE_KEYS_3` instruction in the script, which caused a compilation error. This error is contained in the logged output of the failed test and can be easily fixed by adding the *None* key as the fourth argument.
- Related to previous issue, implemented states for only 3 keys, supporting output of only *A-F*. The issue seems to be caused by the model not fully understanding the logic of the scripting language, rather than simply copying the code for the Morse code processor. This task would require each state to have more transitions and to call the `COMPARE_KEYS_3` instruction repeatedly with different keys, in order to support all of the nine keys. This could be fixed by manually adding the necessary logic, or by well-crafted prompt to explain the possible alternative usage of used features of the scripting language.
- It was expected that the LLM will guess that the step 1 is not needed as the definition *DInputProcessor* already is defined. However, the model created a sub-definition under this one, revealing a bug, where *Core* was not able to detect the root of the *DInputProcessor* definition and thus could not construct a processing pipeline. This was fixed by fixing the bug in the *Core*. Alternatively, single prompt to disregard the newly created definition and use the existing one as the base for the new component variant could be used.

- In the system text, the LLM requested simulating only a *keydown* action, instead of *keypress*. This resulted in the subsequent presses to be interpreted as the key being continuously held down, which the generated component correctly disregarded, as it was expecting a new *keydown* event to trigger the next character. Although using *keypress* is enough to fix this issue, debugging this issue might be difficult for someone not familiar with the data flow of the system.

Although the generated code does have few issues, it still demonstrates clear patterns and structure, which can be easily understood and followed by a developer. Better AI assistant usage, such as better prompt crafting, iterative approach, or multi-agent collaboration, could potentially provide even better results. Although such an approach should not be done in actual development, both for the arguments presented at the end of the section 2.3.2 and for the fact that this was designed to be a complex test task, it shows that the system can be quickly and easily expanded by adding new straightforward and well-defined components.

5.2 Usability Testing

Due to time constraints and limited access to the potential user base, the user experience and usability testing was performed only informally, by few self-selected individuals, including the developer. However, this section will provide some potential experiments and methods for potential future testing of the usability of the system, as well as some insights from the informal testing that was performed. As the system is still in early stages of development, with many user-friendly features not yet fully implemented, such as GUI or filtering functionality based on user groups, it is expected that the usability of the system will improve significantly once these features are implemented.

Because other developers are also a user group for this project, as they may want to expand the system or use it in their own projects, it is important to ensure that the codebase is easy to understand and to work with. The test of LLM compatibility from the previous section can be considered as an indirect test of the developer-friendly design of the codebase. A similar test can also be performed with human developers as well, by giving them a specific task to implement, such as the phone keypad input described in the previous section, and measuring the time taken to implement the given task, how much time was spent reading documentation and understanding the codebase, and how many issues were encountered during implementation. This can be done with developers of different levels of experience, backgrounds, or preferred programming languages, to obtain a broader perspective on the usability of the codebase for different types of developers.

The final important group are the expert users. This includes people who will not be developing any extensions, including the system in their own projects, nor modifying the codebase in any way, but still want to configure the system for their very specific use case and use it on their own terms. Expert users are expected to write their own scripts, modify and create their own configuration files, use advanced console commands for specific configuration or deep inspection of the system, et cetera. Because this group will have very different needs and goals, it is problematic to define a specific test to evaluate the usability of the system for them. However, members of this group can provide valuable subjective feedback on the usability of the system, as well as suggestions for improvement. This can be done through interviews, surveys, or even just informal conversations with potential users.

5.2.1 Proposed Usability Test for Multitasking Potential

The main issue with estimating the usability of the system for boosting multitasking potential is that the user has to learn the control scheme so that they are not distracted during their main task. If the user would use the system for shortcuts, when learning for 30 min a day, it would take a few weeks to become proficient enough to benefit from the system [8]. To learn some alternative writing system, it can take even months of regular practice. To overcome this issue, there are two potential solutions. First, user group can be selected from people who are already proficient in some alternative input method, such as Morse code or binary ASCII. Second, the main and secondary tasks can be made simple enough to be quickly adapted, even if not practical for a real-world use case. For example, assume a simple game-like environment, where the user has to keep a mouse cursor on a randomly moving circle on the screen, while a short random sequence of few characters is displayed above the target which the user has to write this sequence using the system. The writing technique could be simple direct input, pressing predefined key N times to cycle through characters, or selective to switch, for example, between typing and moving mouse.

This simplified test would not measure the potential gain from using this exact system, but would rather measure the potential gain from any system that allows some transformation and routing of user input. The control activity would be to write the provided sequence using a regular keyboard and tracking the target afterward. A user would have some time to achieve the highest possible score in both tasks, for example 2 min. The user would alternate between doing both tasks simultaneously and splitting the time between each task to focus fully on one at a time. The total time when the cursor is over the tracked target during a given amount of time serves as a metric of efficiency of performing the main task, while the error rate and the total amount of written characters over the same amount of time shows the multitasking potential.

If the user can achieve higher scores when performing both tasks simultaneously, compared to performing them separately, that would indicate that the system can indeed increase the potential of multitasking. If a user constantly has a lower score when performing both tasks simultaneously, this would mean that the user most probably will not benefit from using the system, potentially even losing some of the efficiency of performing the main task. If the score is similar in both cases, it still could be worth adapting the system, because user could learn to use the system more efficiently, could find more suitable tasks or means of control. Also, even if no productivity gain is achieved, user could still find the system useful for other reasons. As was discussed in Section 2.5, there are also other important metrics for user input methods besides multitasking potential, such as ergonomics, accessibility, or even just fun and enjoyment. Therefore, even if the system does not boost multitasking potential for a specific user, it could still be useful for them for other reasons.

This test could be modified to be more specific to the intended use case. For example, if the goal is efficient use of a multi-PC setup, the text for the secondary task could be displayed on the second machine, and the user would have to split their attention between two machines. If texting while gaming is the goal, the main task could be playing a simple game, like navigating through a maze, while the secondary task might not be altered. Tracking two different targets on two different machines would emulate tasks such as controlling two live tools at the same time, such as aligning a laser pointer and a mirror for a hypothetical scientific experiment. Another modification could be derived from tasks such as taking notes while using some software. For example, a structural engineer would need to inspect a 3D model of a building, hovering with a mouse over different elements of

the model, while taking notes about simulated loads, stresses, and deflections. A user would first take notes about predefined elements using their preferred method, such as an extra laptop, note-taking software running on the same machine, or a screen sharing software. The total time would then be compared to the time taken to take notes using the system, where a user controls the model on a local machine and takes notes on a remote machine, using the same set of peripheral devices. This general idea can be applied to many different use cases to evaluate the potential to improve multitasking options in different scenarios.

5.2.2 Insights from Informal Testing

The first major obstacle to adapting the system is the lack of a functioning graphical user interface, which is, at the moment of writing, present only as a proof of concept that is awaiting full implementation. While pre-made scenarios under a configuration file is functional and the user has to enter only an initial password and one command to start the system, it still reduced the willingness of users to test the system. This is not surprising, and user-friendly features, such as a GUI or filtering functionality based on user groups, were planned and designed, but could not be implemented in time. However, after providing guidance and occasional assistance, users were able to use the system and test it in real-world scenarios, expressing positive feedback and interest in using the system once it implements more user-friendly features. In particular, the author of the project is already starting to learn custom complex remapping for their personal use.

Outside of the small test group, the idea was enthusiastically received by the gaming friend group, with many considering it to be a potentially useful tool for gaming. Converting these positive reactions into the willingness to test the system and provide feedback proved to be a challenging task, mostly due to the time constraints on both sides. When suggesting the system in online communities related to gaming, the reaction was more mixed. While some users showed some interest, many users seemed to have already found their own ways to make more efficient use of the time spent playing video games. Others expressed concerns about losing immersion in the game or steep learning curve. The original idea was to present a short video of using in-game chat while actively playing a game, which was expected to provide a good demonstration of the potential of the system. But because many of the user-friendly features were not fully implemented, potential users were unable to test the system on their own, and thus all the online discussions stayed at the stage of gathering general ideas and behavior of potential users.

In terms of non-gaming use cases, potential users had to be presented with a very specific example of a use case in order to be able to consider potentially using the system. This was apparent with a scenario very close to the example of a structural engineer from the end of the previous section, where a Synergy based setup was deemed to be an ideal solution, especially compared to a physical KVM switch. A potentially more efficient workflow when using the designed system was considered only after pointing out the need to keep moving mouse cursor between two machines, and the potential of the system to reduce this movement. This indicates that the system can be useful for a broad range of users, but once some workflow is established, it can be difficult to perform a critical introspection and to see a space for improvement.

Chapter 6

Conclusion

The main goal of this project was to design and implement a highly configurable system for a transformation and routing of user input. The focus was put on the aspects specified in Section 3.1, namely security, portability, extensibility, selectiveness, and performance. To achieve this goal, first the theory of user input sources and handling, along with some existing solutions, was explored in Chapter 2. This showed that options for user input can be quite rigid and fragmented, while modern computer setups might often benefit from more flexible and unified control. Existing solutions for input routing and transformation are often limited to specific use cases and prefer limited options rather than risking difficult adaptability or potential security issues.

Based on this theoretical background, the general design of the proposed system was discussed in Chapter 3. Instead of implementing a monolithic application that is fine-tuned for specific use cases that cover as wide range of potential users as possible, a modular framework was proposed, along with an exemplary application built on top of it. The framework is designed to be highly configurable by replacing or adding new components. Provided implementations of these components are designed to be as hardware and operating system independent as possible to allow adaptation for various machines and setups.

To enable the system to be changed and extended during runtime, multiple systems were proposed to manage communication between components, mainly console commands, which can also serve as an API for external applications, pipeline to connect components together. The user is also able to create custom components and console commands as a part of external libraries, which can be loaded during runtime. To enable the provision of custom logic for either system behavior or to define custom input transformations without the risk of giving external code full access to the system, a custom scripting system was proposed. Because the system will probably be used to handle sensitive data, various security measures were proposed in Section 3.5 to inform the user about potential issues and provide options to mitigate them.

To provide a default implementation to offer some user input processing out-of-the-box, the Windows operating system was chosen. Input capturing and simulation is implemented using Low Level Hooks, which provides good compromise to satisfy all required capabilities. The input is immediately parsed into a generic data structure that can be processed without relying on a hardware or operating system. After such parsing, input is expanded by relevant context information and processed based on user defined configuration. This processing is done on a generalized input data structure. With this design, the system can be ported to other operating systems by only changing the components for input capture and simulation.

Although not all proposed features were implemented in time, namely full GUI, native Linux support, user groups, and a larger pool of modules for user scripts, the implemented system provides all the necessary features to be used in real world scenarios. The project is made available under AGPL-3.0 license on GitHub [GitHub](https://github.com/xBecanto/InputResender)¹ as an open source project, and it is planned to be further developed and improved in the future.

Finally, Chapter 5 presents the results of correctness and performance testing of the implemented system. The results show that the system is able to perform all the required tasks with good performance. Usability testing in Section 5.2 suggests that the system can be easily extended with additional components and that the configuration options are sufficient to support various use cases. In addition, additional experiments were proposed to better assess the potential of the system once a larger test group is available.

In general, the proposed system is able to provide a wide range of transformation and routing options, which can be easily configured and extended, and nothing should be preventing the adaptation to various hardware and operating systems. The use of the system does not disrupt the original task flow and can improve multitasking capabilities or user experience. Specifically, the motivational example in the beginning of chapter 2, that is a person often switching between a video game and programming, was tested on a subjective bases with promising results.

¹<https://github.com/xBecanto/InputResender>

Bibliography

- [1] APPERT, C. and ZHAI, S. Using strokes as command shortcuts: cognitive benefits and toolkit support. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. New York, NY, USA: Association for Computing Machinery, 2009, p. 2289–2298. CHI '09. ISBN 9781605582467.
- [2] DIX, A.; FINLAY, J.; ABOWD, G. and BEALE, R. Human-Computer Interaction. In: January 2004. ISBN 9780130461094.
- [3] ENGSTRÖM, J. *Web Development with Blazor*. Birmingham, England: Packt Publishing, june 2021. ISBN 9781800208728.
- [4] GRIGORIAN, A.; FANG, P.; KIRK, T.; EFENDIZADE, A.; JADIDI, J. et al. Learning from Gamers: Integrating Alternative Input Devices and AutoHotkey Scripts to Simplify Repetitive Tasks and Improve Workflow. *Radiographics*, january 2020, vol. 40.
- [5] JACKO, J. and SEARS, A. The Human-Computer Interaction Handbook: Fundamentals, Evolving Technologies, and Emerging Applications. In: September 2002, p. –1312. ISBN 0805838384.
- [6] KHAN, R.; SURTI, A.; REHMAN, R. and ALI, U. Knowledge and practices of ergonomics in computer users. *JPMA. The Journal of the Pakistan Medical Association*, march 2012, vol. 62, p. 213–7.
- [7] KIESEL, A.; JOHANNSEN, L.; KOCH, I. and MÜLLER, H. *Handbook of Human Multitasking*. 1stth ed. Springer, september 2022. ISBN 9783031047602.
- [8] KIM, J. W. and RITTER, F. E. Learning, Forgetting, and Relearning for Keystroke- and Mouse-Driven Tasks: Relearning Is Important. *Human-Computer Interaction*. Taylor & Francis, 2015, vol. 30, no. 1, p. 1–33. Available at: <https://doi.org/10.1080/07370024.2013.828564>.
- [9] KOSMYNA, N.; HAUPTMANN, E.; YUAN, Y.; SITU, J.; LIAO, X.-H. et al. Your Brain on ChatGPT: Accumulation of Cognitive Debt when Using an AI Assistant for Essay Writing Task, june 2025.
- [10] LEOPOLD, J. and AMBLER, A. Keyboardless visual programming using voice, handwriting, and gesture. In: *Proceedings. 1997 IEEE Symposium on Visual Languages (Cat. No.97TB100180)*. 1997, p. 28–35.
- [11] LOCK, A. *ASP.NET core in action, third edition*. New York, NY: Manning Publications, august 2023. In Action. ISBN 9781633438620.

- [12] MALLET, C.; GRAY, S. et al. *AutoHotkey Documentation*. AutoHotkey Community, 2026. Available at: <https://www.autohotkey.com/docs/v1/>. Accessed: 2026-01-16.
- [13] MARKLIN, R. W. and SIMONEAU, G. G. Design Features of Alternative Computer Keyboards: A Review of Experimental Data. *Journal of Orthopaedic & Sports Physical Therapy*, 2004, vol. 34, no. 10, p. 638–649. PMID: 15552709.
- [14] MENEZES, A. J.; OORSCHOT, P. C. van and VANSTONE, S. A. *Handbook of Applied Cryptography: Discrete Mathematics and Its Applications*. CRC Press, december 1996. ISBN 9780849385230.
- [15] MICHAELS, P. *Software Architecture by Example: Using C# and .NET*. Apress Berkeley, CA, april 2022. ISBN 9781484279892.
- [16] MICROSOFT. *LowLevelKeyboardProc function*. Microsoft Docs, 2025. Available at: <https://learn.microsoft.com/en-us/windows/win32/winmsg/lowlevelkeyboardproc#remarks>. Accessed: 2026-01-16.
- [17] MICROSOFT. *Raw Input*. Microsoft Docs, 2025. Available at: <https://learn.microsoft.com/en-us/windows/win32/inputdev/about-raw-input>. Accessed: 2026-01-18.
- [18] MIHAILESCU, M. I. and NITA, S. L. *Pro Cryptography and Cryptanalysis: Creating Advanced Algorithms with C# and .NET*. 2021. ISBN 9781484263679.
- [19] MITANI, M.; SATO, S. and HINOKI, I. *The Manga Guide to Cryptography*. San Francisco, CA: No Starch Press, july 2018.
- [20] MO, R.; WANG, D.; ZHAN, W.; JIANG, Y.; WANG, Y. et al. Assessing and Analyzing the Correctness of GitHub Copilot’s Code Suggestions. *ACM Trans. Softw. Eng. Methodol.* New York, NY, USA: Association for Computing Machinery, august 2025, vol. 34, no. 7. ISSN 1049-331X.
- [21] MRAZEK, K.; HOLTON, B.; KLEIN, T.; KHAN, I.; AYELE, T. et al. The Tap Strap 2: Evaluating Performance of One-Handed Wearable Keyboard and Mouse. In: STEPHANIDIS, C.; KUROSU, M.; CHEN, J. Y. C.; FRAGOMENI, G.; STREITZ, N. et al., ed. *HCI International 2021 - Late Breaking Papers: Multimodality, eXtended Reality, and Artificial Intelligence*. Cham: Springer International Publishing, 2021, p. 82–95. ISBN 978-3-030-90963-5.
- [22] NEMETH, E.; SNYDER, G.; HEIN, T.; WHALEY, B. and MACKIN, D. *UNIX and Linux System Administration Handbook*. 5thth ed. Prentice Hall, august 2017. ISBN 9780134277554.
- [23] NORMAN, D. A. *The Design of Everyday Things*. Basic Books, november 2013. ISBN 9780465050659.
- [24] RASKIN, J. *The Humane Interface: New Directions for Designing Interactive Systems*. Reading, Mass.: Addison-Wesley, 2000. ISBN 9780201379372.
- [25] SARCAR, V. *Simple and Efficient Programming with C#*. 2ndth ed. Apress Berkeley, CA, september 2021. ISBN 9781484273227.

- [26] SHARP, H.; ROGERS, Y. and PREECE, J. *Interaction Design. Beyond Human-Computer Interaction*. January 2007. ISBN 9780470018668.
- [27] SYMLESS. *Synergy*. 2025. Available at: <https://github.com/symless/synergy/releases/tag/v1.20.0>. Accessed: 2026-01-16.
- [28] TEAM, S. *Serenade Documentation*. Serenade Labs, Inc., 2023. Available at: <https://serenade.ai/docs/>. Accessed: 2026-01-16.
- [29] TEILHET, S. *Subclassing & hooking with visual basic*. Sebastopol, CA: O'Reilly Media, june 2001. ISBN 9780596001186.
- [30] TROELSEN, A. and JAPIKSE, P. *Pro C# 10 with .NET 6*. Apress, july 2022. ISBN 9781484278697.
- [31] VIERIU, A. and PETREA, G. The Impact of Artificial Intelligence (AI) on Students' Academic Development. *Education Sciences*, march 2025, vol. 15, p. 343.
- [32] ZHANG, B.; LIANG, P.; ZHOU, X.; AHMAD, A. and WASEEM, M. Practices and challenges of using github copilot: An empirical study. *ArXiv preprint arXiv:2303.08733*, 2023.

Appendix A

Contents of the included storage media

- `/build` — Self-contained application
 - `./InputResender.WindowsGUI.exe` — The main executable file
 - `./InputResender.ExternalExtensions.dll` — Loadable DLL with component for clipboard manipulation
 - `./config.xml` — Configuration file, protected by ‘asdf’ password
 - `./README.md` — Copy of readme for the project
- `/source` — Source code solution
 - `./InputResender.sln` — The main solution for the project
 - `./README.md` — A documentation for the project
 - `./Docs` — Additional documentation files, referenced in the readme file
- `/thesis` — Documents with this text
 - `./source` — Source files for LaTeX project
 - `thesis.pdf` — Electronic version of this thesis
 - `thesisPrint.pdf` — Print version of this thesis